

Linux на встраиваемых устройствах

Лабораторный практикум

Лаборатория «СГУ — Базальт СПО»

24.06.2026

Содержание

1	Лабораторная №1. Освоение базовых навыков работы с одноплатником.	5
1.1	Цель работы	5
1.2	Подготовительный материал	5
1.3	Контрольные вопросы	11
1.4	Задание	11
2	Лабораторная №2. Работа с платой по SSH.	12
2.1	Цель работы	12
2.2	Подготовительный материал	12
2.3	Контрольные вопросы	13
2.4	Задание	14
3	Лабораторная №3. Использование кросс-компилятора.	15
3.1	Цель работы	15
3.2	Подготовительный материал	15
3.3	Контрольные вопросы	18
3.4	Задание	19
4	Лабораторная №4. Создание основных компонентов образа. Ядро Linux.	20
4.1	Цель работы	20
4.2	Подготовительный материал	20
4.3	Контрольные вопросы	25
4.4	Задание	25
5	Лабораторная №5. Создание основных компонентов образа: дерево устройств, корневая файловая система, загрузчик.	27
5.1	Цель работы	27
5.2	Подготовительный материал	27
5.3	rootfs	32
5.4	Контрольные вопросы	33
5.5	Задание	34
6	Лабораторная №6. Подготовка носителя и запись образа на носитель.	35
6.1	Цель работы	35
6.2	Подготовительный материал	35
6.3	Разбивка SD-карты	35
6.4	Запись компонентов образа	36
6.5	Контрольные вопросы	39
6.6	Задание	39
7	Лабораторная №7. Добавление поддержки SPI интерфейса	40

7.1	Цель работы	40
7.2	Подготовительный материал	40
7.3	Постановка задачи	42
7.4	Контрольные вопросы	49
7.5	Задание	50
8	Лабораторная №8. Добавление поддержки I2C интерфейса	51
8.1	Цель работы	51
8.2	Подготовительный материал	51
8.3	Постановка задачи	51
8.4	Контрольные вопросы	56
8.5	Задание	56
9	Лабораторная №9. Работа с логическими анализаторами	58
9.1	Цель работы	58
9.2	Подготовительный материал	58
9.3	Контрольные вопросы	66
9.4	Задание	66
10	Лабораторная №10. Основы программирования микроконтроллеров Arduino. Часть 1.	68
10.1	Цель работы	68
10.2	Подготовительный материал	68
10.3	Практическая часть	73
10.4	Контрольные вопросы	75
10.5	Задание	76
11	Лабораторная №12. Протокол MQTT	79
11.1	Цель работы	79
11.2	Подготовительный материал	79
11.3	Практическая часть	83
11.4	Контрольные вопросы	92
11.5	Задание	92
11.6	Полезные ссылки	93
12	Лабораторная №13. Итоговое проектное задание по вариантам	94
12.1	Цель работы	94
12.2	Порядок выполнения	94
12.3	Вариант 1. Метеостанция — сбор и визуализация данных с датчика BME280	95
12.4	Вариант 2. Пульт управления светодиодами — remote control через MQTT и SPI	96
12.5	Вариант 3. Системный монитор одноплатника — dashboard	98

12.6	Вариант 4. Генератор сигналов с управлением по MQTT — практическая верификация анализатором	100
12.7	Вариант 5. Светодиодный проигрыватель мелодий	101
12.8	Вариант 6. Система мониторинга с оповещением на I2C OLED	103
12.9	Вариант 7. Генератор загрузочных образов — консольный сборщик SD- карты	105
12.10	Вариант 8. Система мониторинга со звуковой сигнализацией	106
12.11	Вариант 9. Самописец данных с хранением на SD-карте (Data Logger) . .	108
12.12	Вариант 10. Игра на реакцию	109
12.13	Контрольные вопросы	111
12.14	Порядок сдачи	111

1 Лабораторная №1. Освоение базовых навыков работы с одноплатником.

1.1 Цель работы

Освоить начальные навыки работы с одноплатными компьютерами: научиться подключаться к плате через последовательный порт и выполнять базовые операции в консольной среде.

1.2 Подготовительный материал

Одноплатный компьютер — это полноценный компьютер, размещённый на одной печатной плате. Он содержит процессор, память, устройства ввода-вывода и другие компоненты, необходимые для работы. В отличие от настольных ПК, одноплатники имеют компактные размеры, низкое энергопотребление и часто используются во встраиваемых системах, IoT-устройствах и образовательных целях.

В нашей лаборатории доступны две модели на базе процессора Allwinner D1:

- Lichee RV Dock
- MangoPi MQ Pro

Allwinner D1 (sun20iw1p1) — первый массовый System-on-Chip от компании Allwinner, использующий архитектуру RISC-V. Он содержит одно ядро XuanTie C906 (RV64GCV), разработанное компанией T-Head.

Обе платы сделаны на одном и том же процессоре и имеют схожую периферию и характеристики. **В первых лабораторных необходимо будет работать именно с Lichee.**

1.2.1 Что такое RISC-V?

Процессорные архитектуры определяют базовый набор команд, который понимает процессор. В современных вычислительных системах существуют два принципиально разных подхода.

CISC архитектура (x86_64):

Архитектура x86_64, используемая в большинстве настольных компьютеров и серверов, относится к классу CISC (Complex Instruction Set Computer).

Перечислим значимые характеристики:

- Богатый набор сложных инструкций
- Отдельные команды могут выполнять комплексные операции
- Архитектура является проприетарной, принадлежит компаниям Intel и AMD
- Требуется лицензионных отчислений за использование

RISC-V архитектура:

Архитектура RISC-V (Reduced Instruction Set Computer) представляет противоположный подход:

- Минималистичный набор простых инструкций
- Каждая команда выполняет одну элементарную операцию
- Сложные операции реализуются последовательностью простых команд
- Архитектура является открытой и свободной для использования

1.2.2 Первый запуск

Довольно очевидно, что для запуска операционной системы нужен носитель с операционной системой (в нашем случае карта памяти) и образ операционной системы, который на нее записан.

На момент выполнения текущей лабораторной работы и то, и другое в готовом виде предоставлено студенту

Обычно производитель платы предоставляет готовый тестовый образ для демонстрации ее возможностей. Но у стороннего разработчика всегда есть возможность произвести собственные преобразования, необходимые для решения специфических задач.

1.2.3 Включение и начало работы с платой

Поскольку нам необходимо работать с платой, а платы у нас достаточно малоресурсные, чтобы работать на графике(хотя Lichee RV ее поддерживает) нам нужно работать через консоль.

Для работы с консолью пригодится **UART-преобразователь**, подключаемый на три контакта GPIO платы. Контакты TX и RX у Lichee RV подписаны на задней стороне платы.

На рисунке 1 показан джампер выбора напряжения на UART-преобразователе:

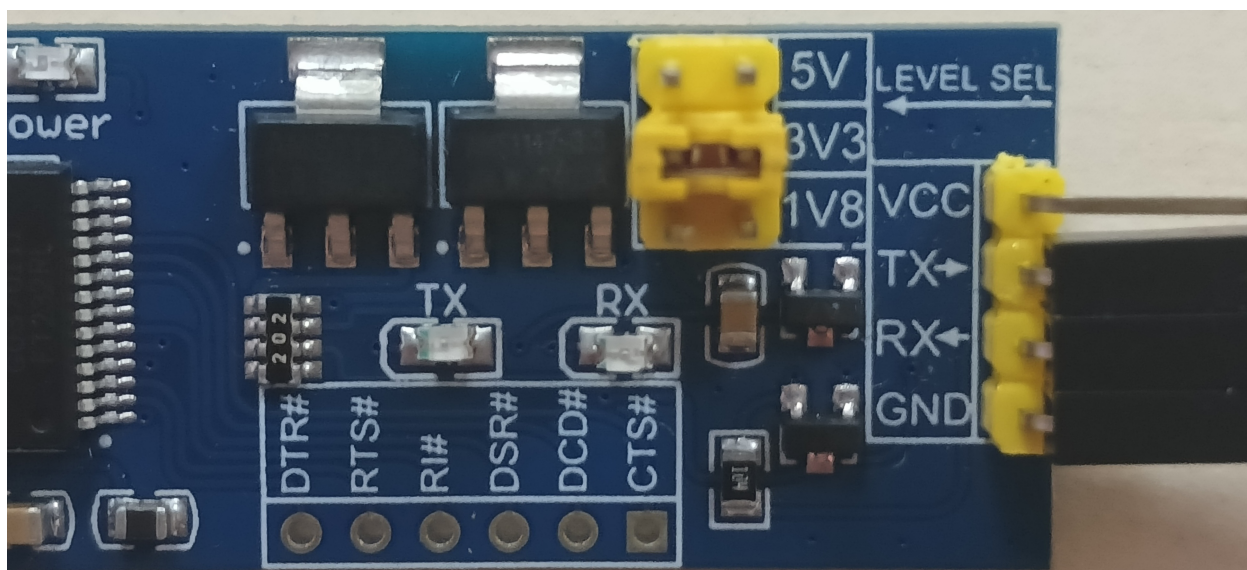


Рис. 1.1 – Джампер

Важно правильно выбрать на UART-преобразователе джампером напряжение. В нашем случае это 3.3V.

Если выбрать напряжение ниже необходимого — передача данных просто не будет возможной, а вот если выбрать выше, то появится хорошая возможность спалить устройство.

ОТНЕСИТЕСЬ К ЭТОМУ ВНИМАТЕЛЬНЕЕ!

Подключаем UART-преобразователь к компьютеру. Для удобства может пригодиться usb-удлинитель из студенческих комплектов, если ПК стационарный.

Далее, соединяем пин GND на GPIO контактах с пином GND на преобразователе, а RX пины соединяем с TX пинами, и наоборот.

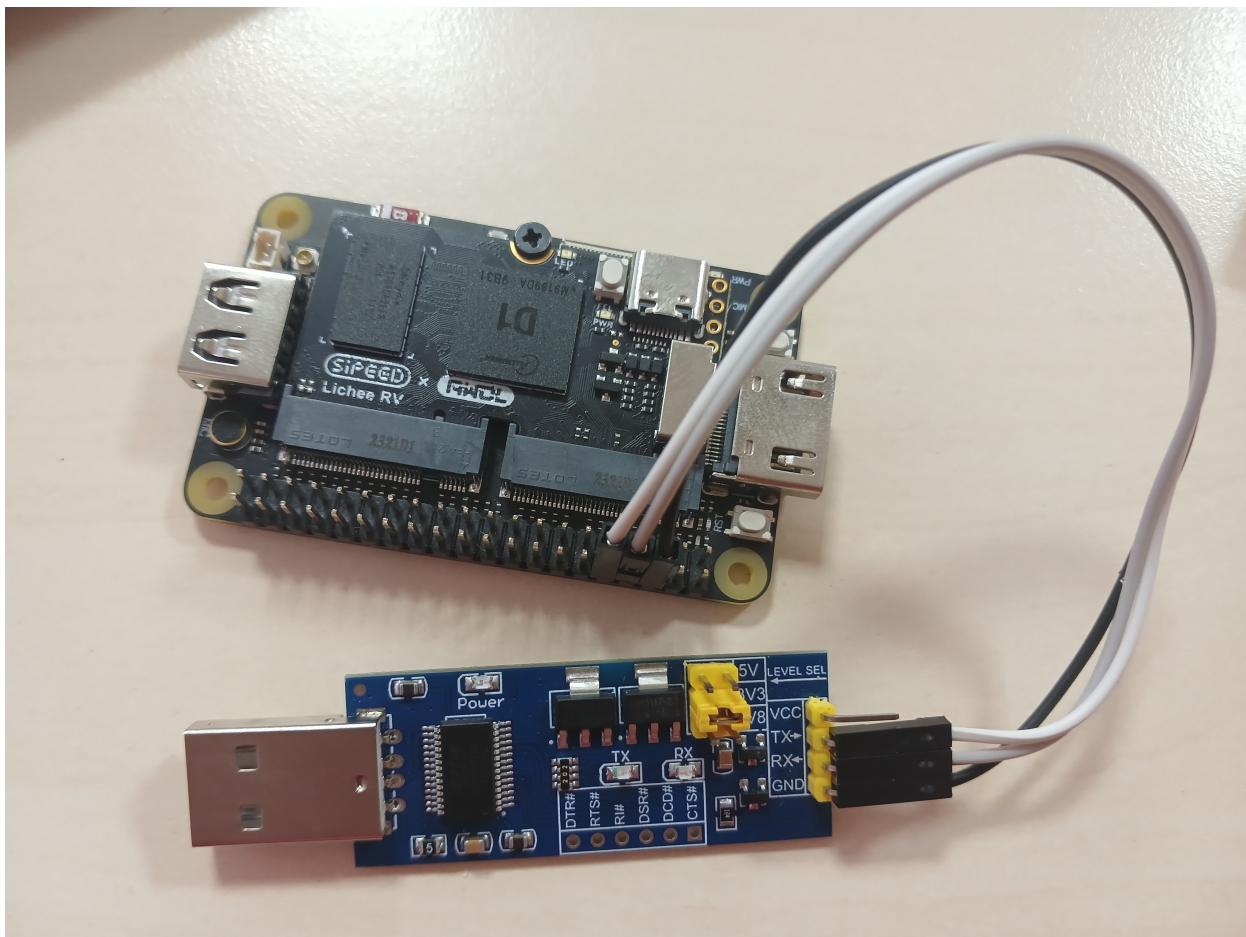


Рис. 1.2 – UART-преобразователь

Далее понадобится программа для взаимодействия с платой. Например можно воспользоваться программой `tio`:

Чтобы определить имя устройства в системе можно почитать логи `dmesg`. Наприме так:

```
# dmesg | grep -i "tty"
```

На рисунке 2 показан вывод команды `dmesg` с именем устройства:

Подключимся к UART-преобразователю, и ждем поступления сигналов

```
$ tio -b 115200 /dev/ttyUSBX # X - число
```



```
[root@taranev ~]# dmesg | grep -i "tty"
[ 0.017205] ACPI: SSDT 0x000000008E82D000 002357 (v02 LENOVO TbtTypeC 00000000 INTL 20200717)
[ 0.157096] printk: legacy console [tty0] enabled
[ 5.476955] systemd[1]: Reached target getty.target - Login Prompts.
[ 27.772101] Bluetooth: RFCOMM TTY layer initialized
[ 763.515842] usb 3-3.1: FTDI USB Serial Device converter now attached to ttyUSB0
[ 765.287605] ftdi_sio ttyUSB0: FTDI USB Serial Device converter now disconnected from ttyUSB0
[ 765.566621] usb 3-3.1: FTDI USB Serial Device converter now attached to ttyUSB0
[ 2422.627797] ftdi_sio ttyUSB0: FTDI USB Serial Device converter now disconnected from ttyUSB0
[ 3108.986077] usb 3-3.4: FTDI USB Serial Device converter now attached to ttyUSB0
[root@taranev ~]#
```

Рис. 1.3 – UART-преобразователь

Теперь если вставить флеш-карточку с записанной на нее образом в соответствующий интерфейс на плате, можно увидеть в консоли логи запуска операционной системы, а затем и приглашение ко вводу.

На всех регулярных сборках ALT регулярных сборках, до прохождения мастера первоначальной настройки, логинка *root* с паролем *altlinux*.

В случае предоставляемого образа, учетные данные root отличаются(см. текст задания)

1.2.4 Короткая справка по утилитам и командам

Обратите внимание, что некоторые команды могут потребовать права суперпользователя.

Информация о процессоре (ЦП) и нагрузка:

`lscpu` # архитектуру, количество ядер, модель процессора

`cat /proc/cpuinfo` # информация о сри

`top` # или `htop` для интерактивного представления

Информация о памяти (RAM) и нагрузка:

`free -h` # общий объем и свободная память

`cat /proc/meminfo` # детальная информация

**** Информация о дисках и пространстве:****

lsblk # Информация о разделах дисков

df -h # Информация о свободном месте

fdisk -l # детальная таблица разделов

Информация о сетевых интерфейсах:

ip a # ip-адреса и состояния интерфейсов

ifconfig # старый аналог

Работа с файлами:

```
mkdir my_folder           # Создать директорию
cd my_folder              # Переход в директорию
touch my_file.txt         # Создать пустой файл "my_file.txt"
echo "Hello, World!" > my_file.txt # Запись текста в файл (перезаписывая)
echo "Lichee" >> my_file.txt # Добавить строку в конец файла
cat my_file.txt           # Вывести содержимое файла в консоль
```

Настройки имени системы и времени:

hostnamectl # информация об имени

timedatectl # информация о времени

Работа с пользователями:

useradd [user_name] # создать пользователя

passwd [user_name] # задать пароль

su - [user_name] # переключиться на другого пользователя

1.3 Контрольные вопросы

1. Чем архитектура RISC-V принципиально отличается от x86_64?
2. Почему для первого подключения к одноплатнику используется UART, а не SSH?
3. Что такое GPIO и какую роль играет распиновка (pinout) при подключении периферии?
4. Как определить, к какому файлу устройства в /dev/ подключился UART-конвертер?
5. Что показывает команда `lscri` на одноплатнике и как интерпретировать поле Architecture для RISC-V?

1.4 Задание

Ознакомившись с подготовительным материалом к лабораторной №1 выполнить следующие подзадачи:

- Завести учетную запись в домене лаборатории
- Удостовериться в возможности авторизации на ПК и возможности пользоваться `sudo`
- Подключить одноплатник к ПК через UART-переходник
- Выполнить подключение к консоли Linux с помощью утилиты `tio`
- Авторизоваться в системе с учетными данными: `root/qwe123`
- Выполнить базовые команды, которые позволят:
 - Узнать текущую нагрузку на ЦП и информацию о нем
 - Узнать текущую нагрузку на RAM и информацию о ней
 - Узнать информацию о дисках
 - Узнать информацию о сетевых интерфейсах
 - Создать директорию, файл, записать в файл текст и вывести его в консоль
 - Узнать настройки имени и времени
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

2 Лабораторная №2. Работа с платой по SSH.

2.1 Цель работы

Освоить методы удаленного взаимодействия с одноплатным компьютером: научиться использовать и настраивать сетевое подключение по SSH, и выполнять передачу файлов между системами.

2.2 Подготовительный материал

Протокол SSH (Secure Shell): SSH — криптографический сетевой протокол, обеспечивающий безопасный обмен данными в незащищенной сети. Основные возможности: - Удаленное выполнение команд - Передача файлов - Туннелирование TCP-соединений - Авторизация с использованием ключей

SCP (Secure Copy Protocol): Протокол для безопасной передачи файлов, использующий SSH для аутентификации и шифрования. Позволяет копировать файлы между системами через командную строку.

2.2.1 Краткие подсказки по удаленному подключению

Стандартное расположение конфигурационного файла протокола SSH на Linux:

```
/etc/openssh/sshd_config
```

В случае работы по ssh с помощью парольной аутентификации может быть достаточно раскомментировать следующие строки конфигурационного файла, установив параметрам нужное значение:

PasswordAuthentication yes

PermitRootLogin yes # в случае если нужно подключиться именно к суперпользователю

После внесения изменений необходимо перезапустить службу sshd:

```
# systemctl restart sshd
```

Подключение к удаленной машине по ssh:

```
ssh [опции] пользователь@хост
```

Передача файла на удаленную машину:

```
scp файл пользователь@хост:путь/на/удаленном/хосте
```

Пример настройки сети на одноплатнике:

```
# nmcli dev show # узнаем как называется wifi-интерфейс
# nmcli dev wifi # узнаем доступные wifi сети
# nmcli dev wifi connect "название_сети" password "пароль_сети"

# ping ya.ru # проверим, что у появился DNS и выход в интернет

# ip a # узнаем текущий айпишник платы
```

2.3 Контрольные вопросы

1. В чём принципиальное отличие SSH от UART-соединения?
2. Какие параметры в /etc/openssh/sshd_config необходимо изменить для разрешения входа по SSH на одноплатнике?
3. Зачем нужна утилита scp и как она работает?
4. Как настроить Wi-Fi-соединение на одноплатнике без графического интерфейса?
5. Почему рекомендуется отключать аутентификацию по паролю и переходить на ключи SSH?

2.4 Задание

Ознакомившись с подготовительным материалом к лабораторным №1 и лабораторным №2 осуществить запуск одноплатника Lichee RV Dock с операционной системой Linux и решить следующие подзадачи:

- Настроить сетевое подключение на одноплатнике (Wi-Fi или Ethernet)
- Установить и настроить ssh на плате
- Подключиться к плате по SSH с рабочего компьютера
- Создать своего пользователя, со своим паролем и в дальнейшем работать через него
- Выполнить передачу файлов между системами с использованием SCP
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

3 Лабораторная №3.

Использование кросс-компилятора.

3.1 Цель работы

Научиться собирать программы для RISC-V на x86_64 машине.

3.2 Подготовительный материал

На момент выполнения лабораторной №3 читатель находится в следующих обстоятельствах:

- есть плата с архитектурой RISC-V, под которую нужно писать программный код;
- есть ПК на архитектуре x86_64, на котором удобно писать программный код;

Возникает довольно обоснованный вопрос: “Как организуют работу с кодом люди, которые постоянно разрабатывают программный код, поставляемый на процессорные архитектуры отличные от x86_64”.

Общий принцип всегда один:

- пишется код
- компилируется кросс-компилятором
- так или иначе переносится на целевую платформу ИЛИ целевая платформа эмулируется
- код запускается и отлаживается

Опустим процесс написания кода и рассмотрим остальные этапы этого принципа.

3.2.1 Кросс-компиляция

Для компиляции программы на хостовой машине под целевую платформу необходимо использовать специальный **тулчейн**, который может быть установлен вместе с пакетом *gcc-riscv64-linux-gnu*.

Тулчейн (Toolchain) — это набор инструментов (утилит, компиляторов, библиотек), которые используются для разработки и сборки программного обеспечения под определенную платформу или архитектуру.

Главное, что оказывается необходимым — **кросс-компилятор**.

Кросс-компилятор — это специальный компилятор, который позволяет компилировать код для платформы, отличной от той, на которой он выполняется.

Где можно найти кросс-компилятор? Например в репозитории для разработчиков Альт Линукс!

Настройка репозитория пакетов:

Классически в дистрибутивах Alt все пакеты устанавливаются из репозитория, которые поддерживаются сообществом ALT(Alt Linux Team).

Узнать, о том, из каких репозитория пакетный менеджер apt выкачивает программные компоненты можно с помощью следующего вызова:

```
apt-repo
```

Зачастую многие инструменты необходимые для разработки находятся в специальном репозитории под названием Sisyphus. Для переключения на него(если ваша машина еще не базируется на нем) нужно выполнить от root:

```
apt-repo rm all
```

```
apt-repo set sisyphus
```

```
apt-get update
```


Установка кросс-компилятора:

Настроив работу с репозиторием на машине, специальный тулчейн с кросс-компилятором можно установить одной программой:

```
# apt-get install gcc-riscv64-linux-gnu
```

Итак. Кросс-компилятор — есть.

3.2.2 Среда для написания кода

Нужно решить, где писать программный код.

Классический опыт написания кода связан с разработкой с помощью IDE. Как показывает практика, в сфере embedded-разработки этот подход является менее популярным. По сути IDE является всего лишь надстройкой(пускай зачастую вполне удобной и полезной) над консольными командами, которые выполняют обращения к компиляторам, интерпретаторам и отладчикам с определенными параметрами в зависимости от настроек, установленных пользователем.

Зачастую многим разработчикам привычно пользоваться популярными редакторами такими как vim, neovim, nano и emacs.

В рамках курса вероятно студентом будет затронуто оба варианта написания кода.

В рамках данной лабораторной будет достаточно воспользоваться любым из перечисленных редакторов.

3.2.3 Пример компиляции и тестовый запуск

Скомпилировать исполняемый файл на хостовой машине можно так:

```
riscv64-linux-gnu-gcc -o название_исполняемого_файла файл_исходник.c [библи
```

Кроме того, альтернативным вариантом может быть компиляция непосредственно на целевой плате или использование технологии `qemu-user-static-binfmt`.

`qemu-user-static-binfmt` — это механизм, который позволяет запускать программы, собранные для архитектуры (например RISC-V), напрямую на компьютере с другой архитектурой.

Когда пользователь пытается выполнить файл, собранный для RISC-V, ядро Linux анализирует его заголовок и благодаря настроенному `binfmt_misc` понимает, что данный файл не может быть выполнен напрямую, но для него зарегистрирован интерпретатор. Этим интерпретатором выступает статически скомпилированный `qemu-riscv64-static`, который автоматически запускается ядром и получает целевой исполняемый файл в качестве аргумента.

Далее всю работу берет на себя эмулятор QEMU. Он читает машинный код RISC-V, транслирует его в инструкции, понятные реальному процессору хостовой системы, и выполняет их. При этом, если эмулируемая программа обращается к операционной системе с каким-либо запросом, например на открытие файла или вывод текста, QEMU перехватывает этот системный вызов, преобразует его в соответствующий вызов для архитектуры хоста и передает ядру Linux. Таким образом, программа на RISC-V получает доступ ко всем ресурсам системы, не подозревая, что работает через прослойку.

Важно: Для корректной работы на компьютере исполняемый файл необходимо компилировать с помощью кросскомпилятора статически (с использованием параметра `-static`). В противном случае QEMU не будет знать по умолчанию, где он может взять динамическую библиотеку для работы вашего кода.

Достаточно установить пакет из репозитория Sisyphus:

```
# apt-get install qemu-user-static-binfmt-riscv
```

И попробовать выполнить скомпилированный ранее файл.

3.3 Контрольные вопросы

1. Что такое кросс-компиляция и в каких случаях она необходима?

2. Зачем нужен флаг `-static` при линковке программ для RISC-V?
3. Какую роль выполняет `qemu-user-static` и `binfmt_misc` в процессе кросс-компиляции?
4. Чем отличается тулчейн `gcc` от `riscv64-linux-gnu-gcc`?

3.4 Задание

Ознакомившись с подготовительным материалом к лабораторной №3 выполнить следующие подзадачи:

- Убедиться в наличии или установить кросс-компилятор и подходящий вам редактор на хостовой машине
- Написать приложение, которое с помощью библиотеки `math` высчитывает 5 значений любых элементарных математических функций для аргумента передаваемого приложению через CLI (см. пример работы ниже)
- Скомпилировать программу кросс-компилятором
- Попробовать запустить программу с помощью `qemu-user-static-binfmt`
- Передать исполняемый файл на плату и выполнить его
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач `.doc` и **выслать на почту преподавателя до обозначенного срока**

На рисунке ниже показан пример работы собранного приложения, запущенного через QEMU user-static:

```
[taranev@taranev work_repos]$ ./test_cli 2
Math calculations for x = 2.0000:
=====
sqrt(x)      = 1.414214
x^2           = 4.000000
x^3           = 8.000000
sin(x)        = 0.909297
cos(x)        = -0.416147
```

Пример работы приложения

4 Лабораторная №4. Создание основных компонентов образа. Ядро Linux.

4.1 Цель работы

Изучить что такое ядро ОС и за что оно отвечает, научиться изменять конфигурационный файл для сборки ядра и собрать ядро с заданной конфигурацией.

4.2 Подготовительный материал

4.2.1 Ядро и его сборка

Ядро Linux — это самая главная и центральная часть операционной системы Linux.

Если представить ОС в виде слоёв, то ядро находится в самом центре, между оборудованием компьютера (процессор, память, диски) и всеми остальными программами.

Обозначим основные задачи:

- управление аппаратными ресурсами
- обеспечение безопасности и разграничение прав;
- обеспечение работы сетевых протоколов;
- управление файловыми системами, обеспечение чтения и записи данных на диски.

Стоит отметить, что само по себе ядро - это не полноценная ОС. Комбинация ядра Linux и набора программ из проекта GNU (компиляторы, библиотеки, системные утилиты) образует ту самую операционную систему, которую мы называем Linux.

Сборка ядра Linux из исходных кодов — это процесс компиляции и компоновки исходного кода ядра в исполняемые файлы (ядро и загружаемые модули) на основе предоставленной вами конфигурации.

4.2.2 Подготовка к сборке ядра

Предварительно, установим необходимые пакеты:

```
# apt-get install wget build-essential bc flex bison ncurses-  
devel openssl libssl-devel dtc
```

Архив с исходным кодом получим в папке */usr/src/kernel/sources/kernel-source-6.16.tar*, установив соответствующий пакет.

Для этого переключимся на репозиторий Sisyphus и выкачаем из него архивы с пакетами.

```
# apt-repo rm all
```

```
# apt-repo set sisyphus
```

```
# apt-get update
```

А затем установим нужный пакет

```
# apt-get install kernel-source-6.16
```

Распакуем в удобное место архив и перейдем в его корневую папку.

Далее, нам будет необходимо сконфигрировать файл *.config*, который собирает в себя все возможные параметры для сборки ядра.

Существует несколько способов создания такого файла. Воспользуемся классическим интерфейсом для *make*. Выполним

```
$ make menuconfig
```

Откроется интерактивный интерфейс для тонкой настройки, который позволяет представить древовидную структуру всех опций ядра (драйверы устройств, поддержка файловых систем, сетевые функции и т.д.) и выбирать нужные.

В `menuconfig` все навигируется стрелками. Выбор опций осуществляется с помощью `Enter`, включение (Y), выключение (N), установка опции как модуль (M) осуществляется с помощью соответствующих клавиш. Кроме того может пригодиться поиск нужных опций конфигурации и их зависимостей (/).

По завершении выбора нужных настроек все сохраняется в файл конфигурации `.config`.

4.2.3 Пример изменения конфигурации

Приведем **пример** включения параметра `PREEMPT_RT`, отвечающего за включение в ядро модуля поддержки мягкого реального времени.

Для начала, можно в файл `.config` включить стандартную конфигурацию ядра, которая заполнит все параметры и теоретически позволяет запуститься ядру на любом устройстве указанной архитектуры. Сделать это можно так.

```
$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig
```

Эта команда готовит исходный код ядра Linux к сборке для архитектуры RISC-V, используя кросс-компиляцию. Рассмотрим подробнее, что мы передали в качестве аргументов и что будет выполнено в результате.

ARCH=riscv указывает целевую архитектуру процессора, для которой мы собираем ядро. Исходный код ядра Linux поддерживает десятки архитектур (`x86_64`, `arm`, `arm64`, `powerpc`, `riscv` и т.д.). Код, специфичный для каждой архитектуры, лежит в разных поддиректориях (например, `arch/x86/`, `arch/arm/`, `arch/riscv/`). Переменная `ARCH` говорит утилите `make` и системе сборки, к какой из этих директорий обращаться для поиска правильных исходников, конфигураций и правил сборки.

В данном случае мы говорим системе — “собирай не для моей родной архитектуры (скорее всего, x86_64), а для RISC-V”.

CROSS_COMPILE=riscv64-linux-gnu- задает префикс для набора инструментов компилятора (toolchain), который используется для кросс-компиляции.

Для этого нужен специальный компилятор — кросс-компилятор. Его бинарные файлы обычно имеют префикс, указывающий на целевую архитектуру.

riscv64-linux-gnu- — это и есть тот самый префикс. Система сборки будет вызывать не просто gcc, а riscv64-linux-gnu-gcc; не ld, а riscv64-linux-gnu-ld и так далее. Об установке нужного кросс-компилятора прочитать можно [здесь](#).

defconfig — это цель (target) для make. Она генерирует базовый конфигурационный файл (.config) для выбранной архитектуры.

В директории arch//configs/ (в нашем случае arch/riscv/configs/) лежат несколько заранее подготовленных конфигурационных файлов. defconfig — это обычно конфигурация по умолчанию для данной архитектуры. Она включает все необходимые опции для базовой работоспособности ядра на большинстве устройств с этой архитектурой, но без специфичных драйверов для какого-то конкретного железа.

Выполнение этой цели скопирует конфиг из arch/riscv/configs/defconfig в корневую директорию с исходным кодом ядра и сохранит его как файл .config, что и станет отправной точкой для нашей настройки.

Произведем донастройку получившейся конфигурации.

```
$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- menuconfig
```

P.S указание архитектуры здесь необходимо для того, чтобы система сборки обратилась к директории arch/riscv/ для получения всех архитектурно-зависимых настроек, списков плат и драйверов.

Итак, мы в графическом меню, и нам необходимо включить параметр *PREEMPT_RT* для того, чтобы добавить соответствующий модуль ядра в конфигурацию. Жмем (/) и пишем название параметра в появившееся поле ввода.

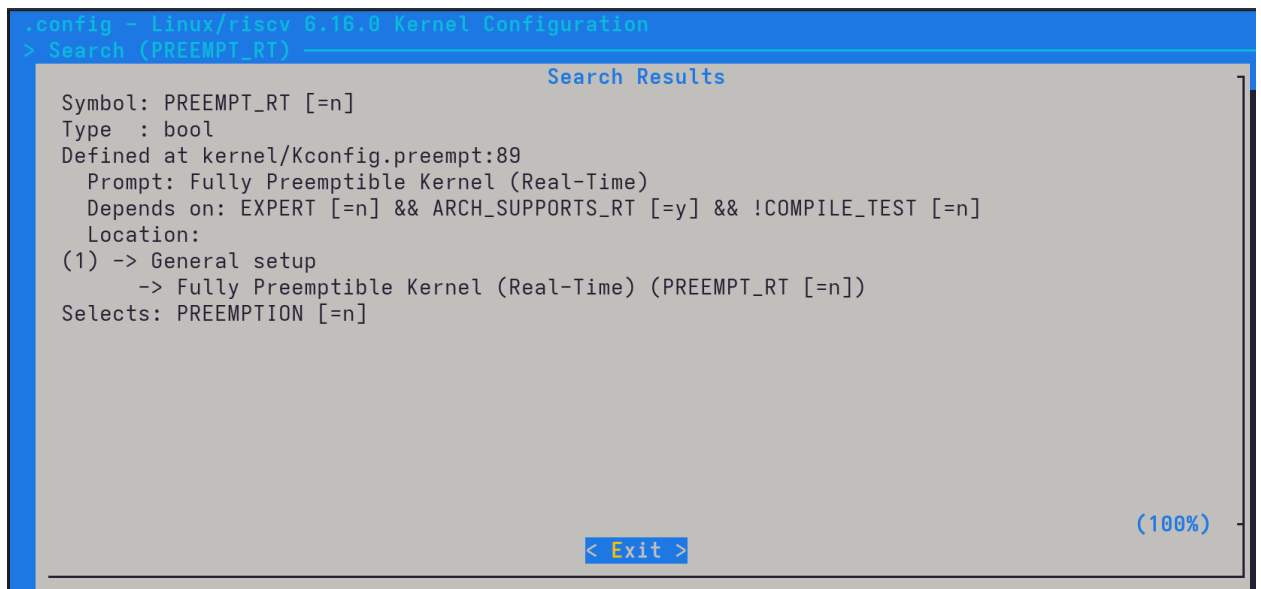


Рис. 4.1 – Поиск параметра `PREEMPT_RT` в `menuconfig`

Получив такой вывод можно сделать вывод о том, где в дереве расположен параметр и какие параметры с какими значениями нужно установить для возможности включения целевого параметра.

Важно отметить, что в окне поиска (в которое можно попасть нажав клавишу (/)) есть возможность быстро перейти к параметру нажатием на клавиатуре (числа) расположенного слева от пути до его расположения. Если параметр в данный момент недоступен для изменения по причине неудовлетворенных зависимостей от других параметров, будет осуществлен переход по той части пути, которая доступна на данный момент.

В нашем случае необходимо чтобы параметр `EXPERT` был включен (он выключен), параметр `ARCH_SUPPORTS_RT` был включен (уже), а параметр `COMPILE_TEST` был отключен (тоже уже так).

Включив параметр `EXPERT` и найдя по указанному пути расположение параметра `PREEMPT_RT` мы получим желаемое.

По завершении указанных действий, можно произвести попытку запуска сборки командой.

```
$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j$(nproc)
```

Обобщенный порядок действий для нашего случая можно описать так:


```
$ cd ~/директория/сборки/

# загрузили в .config конфиг по умолчанию
$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig

# сделали специфичные изменения
$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu menuconfig

# запустили сборку ядра

$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j$(nproc)
```

После окончания сборки по пути *arch/riscv/boot/* можно будет найти бинарный файл готового ядра, а также его сжатый вариант с расширением *.gz*. Кроме того в других подкаталогах будут сгенерированы внешние модули ядра.

Важно: помимо ядра важными файлами, которые необходимо сохранить для выполнения следующих лабораторных работ являются: файл *.config* (находится в директории сборки) и файл сгенерированного дерева устройств (*.dtb*) для одноплатника Lichee RV Dock, путь к которому относительно директории сборки *arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb*

4.3 Контрольные вопросы

1. Какие основные функции выполняет ядро Linux в системе?
2. Для чего используется *make menuconfig* при сборке ядра?
3. Почему при сборке ядра необходимо указывать *ARCH=riscv* и *CROSS_COMPILE=riscv64-linux-gnu-*?
4. Что такое *PREEMPT_RT* и в каких сценариях он необходим?
5. Как правильно выбрать целевой SoC (Allwinner D1) в *menuconfig*?

4.4 Задание

Ознакомившись с подготовительным материалом к лабораторным №4 решить следующие подзадачи:

- Подготовить или убедиться в наличии необходимых для сборки ядра зависимостей
- Сформировать конфигурационный файл для ядра под riscv-архитектура со стандартными параметрами конфигурации
- Внести изменения в полученный конфигурационный файл:
 - По примеру из подготовительных материалов добавить поддержку патча PREEMPT_RT в конфигурацию
 - В разделе SoC selection в качестве целевого варианта выбрать семейство на базе Allwinner D1
 - Включить поддержку wifi-модуля RTW88_8723DS
- Зафиксировать разницу между полученной и стандартной конфигурацией(например, с помощью утилиты diff)
- Добавить любой параметр в конфигурацию, убедившись в его уникальности внутри подгруппы
- Скомпилировать ядро с полученной конфигурацией
- **Сохранить конфигурационный файл и скопированное ядро** для дальнейших лабораторных работ
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

5 Лабораторная №5. Создание основных компонентов образа: дерево устройств, корневая файловая система, загрузчик.

5.1 Цель работы

Ознакомиться с оставшимися необходимыми компонентами образа и подготовить их к записи на носитель: дерево устройств, загрузчик, extlinux.

5.2 Подготовительный материал

5.2.1 Дерево устройств

После завершения сборки, в зависимости от того, поддержка каких SoC была выбрана в соответствующем разделе дерева параметров в директории сборки по пути *arch/архитектура/boot/dts/производитель* из файлов .dts будут скомпилированы файлы .dtb.

Статья на altlinux.org с более подробным описанием.

Введем несколько определений.

Device Tree (DT) — это структура данных, которая описывает “железо” системы для операционной системы. Позволяет одному ядру ОС работать на разных устройствах без перекомпиляции.

DTS (Device Tree Source) — исходный, человекочитаемый текстовый файл с описанием устройства (.dts или .dtsi для include-файлов).

DTB (Device Tree Blob) — бинарный, скомпилированный файл (.dtb), который загружается ядром ОС.

Основной источник(ресурс) dts — исходники ядра Linux и репозитории разработчиков железа. Файлы .dts лежат по пути arch/архитектура/boot/dts/производитель/ (например, arch/riscv/boot/dts/allwinner).

В случае Lichee RV Dock относительный путь из директории сборки к исходникам дерева ядра: *arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dts*

Как такое написать самому:

Если вдруг в наличии подходящего .dts файла нету, то нужно будет найти и изучить Datasheet для вашего SoC. Это главный источник информации об адресах периферии и их регистрах. (само собой эта весьма нетривиальная задача, требующая знания схемы и распиновки устройства, и вряд ли она встанет перед тем, кто портирует линукс на плату выпускаемую крупным производителем).

Вариант упрощения такой задачи - найти .dts файл для максимально похожего устройства в исходниках ядра и его модифицировать под свои нужды.

Поскольку в нашем распоряжении плата Lichee RV **Dock**, и в исходниках ядра есть device tree от производителя, проблем с написанием исходного файла нет.

Остается лишь после сборки ядра взять необходимый .dtb файл из пути *../директория_сборки/arch/riscv/boot/dts/allwinner/*

5.2.2 Загрузчик

Загрузчик (bootloader) — это небольшая программа, которая запускается сразу после включения устройства и отвечает за базовую инициализацию железа (процессор, память, диски), поиск и загрузку в память основной операционной системы (ядра Linux), передачу управления этой операционной системе.

Для одноплатников самым распространенным загрузчиком является Das U-Boot(или просто U-Boot).

Стоит описать схему запуска одноплатника по пунктам:

Включение питания::

Процессор “просыпается” и начинает выполнять код из зашитой в чип памяти (ROM). Этот код очень простой и его задача — найти и запустить следующий этап загрузки (обычно с SD-карты или флешки).

Этап 1: U-Boot::

Первым делом запускается U-Boot. Часто он разделен на две части:

SPL (Secondary Program Loader): Миниатюрная версия U-Boot, которая инициализирует ОЗУ (DRAM). Без этого нельзя загрузить полную, “тяжелую” версию загрузчика.

U-Boot Proper: Полноценный U-Boot, загруженный в память. Он инициализирует периферию (USB, сеть, SD-карту и тд.), находит на файловой системе образ ядра Linux, device tree (файл с описанием “железа” платы) и образ OpenSBI (обычно fw_dynamic.bin).

Этап 2: OpenSBI::

OpenSBI (Open Supervisor Binary Interface) — это реализации спецификации SBI (Supervisor Binary Interface). Этот стандарт определяет интерфейс между программной средой выполнения(M-mode) и супервизором(S-mode).

OpenSBI работает в привилегированном режиме M-mode. Он выполняет финальную низкоуровневую инициализацию процессора (например, настраивает обработку прерываний) и готовит окружение для запуска ядра ОС в менее привилегированном режиме (S-mode).

Этап 3: Передача управления ядру Linux::

OpenSBI выступает в роли “доверенного посредника” (supervisor). Он переключает процессор в режим S-mode и передает управление ядру Linux, которое было загружено U-Boot’ом.

При этом OpenSBI остается в памяти. Когда ядру Linux нужно выполнить привилегированную операцию (например, перезагрузить систему), оно не делает это само, а делает системный вызов (ecall) к OpenSBI, который уже выполняет нужное действие на аппаратном уровне.

Зачастую, u-boot для устройства в том или ином виде создает производитель платы. Это может быть, как готовый u-boot, так и исходники, которые нужно собрать.

Сборка загрузчика:

На неофициальном сайте сообщества linux-sunxi предложен простой вариант сборки из сходников. Процесс сборки в данном случае не будет ничем отличаться от сборки ядра с помощью make.

Все что необходимо сделать:

1. Собрать OpenSBI

1. Склонировать репозиторий для сборки;
2. Установить стандартную конфигурацию для интересующей платы;
3. Запустить сборку.

2. Собрать u-boot

1. Склонировать репозиторий для сборки;
2. Установить стандартную конфигурацию для интересующей платы(возможно, внеся в нее какие то дополнительные изменения);
3. Запустить сборку, сославшись на OpenSBI, с которым будет работать u-boot.

Приведем пример выполнения перечисленных действий.

Соберем OpenSBI:

```
$ git clone https://github.com/riscv-software-src/opensbi
$ cd opensbi
$ make CROSS_COMPILE=riscv64-linux-gnu- PLATFORM=generic FW_PIC=y
```

Соберем u-boot:

```
$ git clone https://github.com/smaeul/u-boot -b dl-wip
$ cd u-boot
$ make CROSS_COMPILE=riscv64-linux-gnu- lichee_rv_dock_defconfig
$ make CROSS_COMPILE=riscv64-linux-gnu- OPENSBI=../opensbi/build/platform/ge
```

После завершения сборки бинарный файл загрузчика *u-boot-sunxi-with-spl.bin* окажется в директории сборки. Останется только записать его на sd-карту, о чем будет рассказано позже.

Можно пойти и по другому пути: взять уже готовый u-boot с OpenSBI версии 1.7 из репозитория.

Для этого переключимся на репозиторий `sisyphus_riscv64`, если сейчас он не выбран

```
# nano /etc/apt/sources.list
```

И вместо текущего содержимого вставляем

```
rpm [sisyphus-riscv64] http://ftp.altlinux.org/pub/distributions/ALTLinux/po
```

```
rpm [sisyphus-riscv64] http://ftp.altlinux.org/pub/distributions/ALTLinux/po
```

Скачаем архивы пакетов и установим нужный пакет

```
# apt-get update
```

```
# apt-get install u-boot-sunxi-riscv
```

Загрузчики для устройств на базе Allwinner D1 после этого окажутся по пути `/usr/share/u-boot`.

Если после этого пакетная база репозитория вам не нужна, можно вернуться с помощью `apt-гиро` к архивам вашей используемой на основной системе ветки репозитория.

5.2.3 Extlinux configuration

На обычном компьютере железо при включении запускает код из BIOS/UEFI, который умеет читать диск, находить загрузчик (например GRUB) и передавать ему управление.

Напомним, что после включения питания сначала загрузочный код в ПЗУ (ROM) на самом процессоре считывает первый раздел на SD-карте, который должен быть в формате FAT32 и ищет на этом разделе специальные файлы вторичного загрузчика. А дальше уже этот более “умный” загрузчик инициализирует железо (видеокарту, память и т.д.) на более высоком уровне.

А вот теперь ключевой момент. После того как загрузчик сделал свою работу, ему нужно передать управление ядру Linux. Но откуда он знает, какое именно ядро запускать и с какими параметрами?

Вот для этого и появился стандарт U-Boot и, в частности, его подсистема “Extlinux Configuration”.

Так вот **extlinux.conf** на одноплатнике — это простой текстовый файл, который сообщает загрузчику U-Boot следующую критически важную информацию:

- Расположение ядра или сжатого ядра
- Какой Device Tree Blob (.dtb) файл использовать
- Параметры командной строки с которым запустить выбранное ядро

Приведем самый простой пример такой конфигурации:

```
label Linux
kernel /Image
fdt /sun20i-d1-lichee-rv-dock.dtb
append root=/dev/mmcblk0p2 rootwait console=ttyS0,115200
```

Кратко расскажем про указанные параметры:

- *kernel /Image* — путь к ядру на разделе BOOT
- *fdt /sun20i-d1-lichee-rv-dock.dtb* — путь к .dtb на разделе BOOT
- *root=/dev/mmcblk0p2* — указывает, где искать корневую файловую систему
- *rootwait* — ждет, пока SD-карта инициализируется, без этого может не найти rootfs
- *console=ttyS0,115200* — нужен, чтобы получать отладочную информацию о запуске через UART

5.3 rootfs

Rootfs (Root Filesystem) — это корневая файловая система. Это основа, с которой начинает работать любая Unix-подобная операционная система (включая Linux) после загрузки ядра.

Проще говоря, это главный иерархический каталог, который содержит всё необходимое для работы системы: исполняемые программы(/bin /sbin /usr/bin), библиотеки(/lib /usr/lib), конфигурационные файлы(/etc), виртуальные файлы устройств(/dev), временные файлы(/tmp), точки монтирования(/mnt /run/media), домашние каталоги пользователей(/home) и многое другое.

На одноплатном компьютере rootfs выполняет абсолютно те же важные функции, что и на сервер или ПК.

- Запуск первой программы (init) и пользовательских приложений.

После того как ядро Linux загружено загрузчиком (U-Boot) и распаковано в память, оно ищет в rootfs программу для запуска под номером 1. Эта первая программа запускает все остальные сервисы, демоны, и приложения.

- Предоставление среды для работы.

Без корневой файловой системы у ядра не будет доступа к критически важным библиотекам (например, libc), конфигурационным файлам (сетевым настройкам, паролям) и самим программам.

Ядро само по себе — просто механизм, менеджер ресурсов. А вся функциональность и пользовательский интерфейс содержатся в rootfs.

- Управление системой и её конфигурация.

Все настройки одноплатника (сетевые адреса, параметры запуска, пароли пользователей и многое другое) хранятся в файлах внутри rootfs (в основном в /etc). Чтобы что-то изменить в работе системы, нужно изменить содержимое rootfs.

Для получения подходящего корневого каталога можно перейти на страницу регулярных сборок altlinux для riscv64 и выбрать подходящий под возможности платы тарболл.

Предлагается остановиться на тарболле с xfce.

5.4 Контрольные вопросы

1. Что такое Device Tree и какую роль он выполняет при загрузке Linux?
2. Чем отличается файл .dts от .dtb?
3. Зачем нужен загрузчик U-Boot? Какие этапы загрузки он проходит?
4. Какую роль выполняет OpenSBI в цепочке загрузки RISC-V?
5. Какая информация содержится в файле extlinux.conf?

5.5 Задание

Ознакомившись с подготовительным материалом к лабораторным №5 решить следующие подзадачи:

- Собрать загрузчик для Lichee RV Dock и скачать аналогичный загрузчик из репозитория Sisyphus-riscv64
- Скачать с указанного преподавателем ресурса готовый архив с rootfs
- Подготовить файл extlinux configuration, указав себя в названии как создателя будущего образа
- **Сохранить перечисленные компоненты**
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

6 Лабораторная №6. Подготовка носителя и запись образа на носитель.

6.1 Цель работы

Освоить процесс подготовки загрузочного носителя для одноплатного компьютера, используя готовые компоненты образа Linux.

6.2 Подготовительный материал

6.3 Разбивка SD-карты

После подготовки компонентов можно описать процесс записи образа на sd-карту, заодно заново перечислив все необходимое.

Итак, начнем с того, что предварительно нам нужно будет изменить разбивку sd-карты, если такая присутствует.

Для этого предварительно затрем начало SD-карты нулями, при чем с запасом, чтобы гарантированно уничтожить таблицу разделов(10 МБ для этого вполне достаточно).

```
$ sudo dd if=/dev/zero of=/dev/sdX bs=1M count=10
```

Продолжим. Запустим утилиту fdisk и зададим разбиение на 2 раздела. После чего, я поясню наглядно, чего мы тем самым добились.

```
$ sudo fdisk /dev/sdX
```

```

o          # создаем новую DOS таблицу разделов
n          # новый раздел
p          # primary
1          # номер раздела
2048       # начальный сектор
+100M      # размер 100MB
n          # новый раздел
p          # primary
2          # номер раздела
[Enter]    # начать сразу после первого раздела
+29.6G     # все оставшееся пространство
w          # записать изменения и выйти

```

Далее нужно отформатировать созданные разделы в файловую систему(vfat) и файловую системы ext4 с соответствующими метками. Выбор файловых систем обусловлен тем, что u-boot “из коробки” работает с файловой системой vfat, а ext4 просто достаточно надежная и подходит для размещения на нем rootfs.

```
$ sudo mkfs.vfat -n B00T /dev/sdX1
```

```
$ sudo mkfs.ext4 -L R00TFS /dev/sdX2
```

Готово!

Можно проверить результат

```
$ sudo fdisk -l /dev/sdX
```

6.4 Запись компонентов образа

Представим схематично, как будет выглядеть финальное пространство sd-карточки. В нашем случае, она на 32 Гб.

Теперь нужно пояснить, что и куда будет записано.

Пространство SD-карты



Рис. 6.1 – Схема разбивки SD-карты на разделы BOOT и ROOTFS

6.4.1 Запись u-boot

Для начала, запишем u-boot в начало карты командой

```
$ sudo dd if=/путь/до/загрузчика/lichee rv dock/u-boot-sunxi-with-spl.bin of=/dev/sdX bs=1k seek=8
```

Таким образом, загрузчик будет записан до начала таблицы разделов, и первоначальный загрузчик платы сможет передать ему управление. Для этого мы специально пропустили с помощью параметра *seek* 8 килобайт памяти от начала пространства карточки. Синим на картинке изображены пропущенные 8K, а красным пространство занимаемое загрузчиком.

В картинке **есть некоторая неточность**, ведь неспроста сначала стиралась а потом заново составлялась **таблица разделов**, которая очевидно присутствует где то на sd-карте. И это действительно так. Между пространством под загрузчик(красный блок) и пространством загрузочного раздела(желтый блок) есть небольшое пространство, в котором и находится таблица разделов.

6.4.2 Раздел BOOT

Для начала необходимо заполнить всем необходимым раздел BOOT

Для начала, положим на BOOT самое главное — ядро(или сжатое ядро .gz) с именем Image(или любым другим). **Перейдем в скопированную директорию** и выполним

```
$ cp kernels/Image_6.16_wifi_rt /путь/до/BOOT/Image
```

Далее, положим туда же скомпилированный файл device tree, полученный в результате выполнения лабораторной №4.

```
$ cp dts_and_dtb/sun20i-d1-lichee-rv-dock.dtb /путь/до/BOOT/
```

Далее, нужно взять полученный в результате выполнения лабораторной №5 конфигурационный файл для загрузчика, который сообщит ему, какое ядро запускать, с каким device tree и какими параметрами. Важно разместить этот файл в директории с одноименным названием *extlinux*.

```
$ cp -r extlinux.conf /run/media/username/BOOT/extlinux
```

6.4.3 Раздел ROOTFS

На этот раздел нам нужно распаковать содержимое, которое полностью соответствует заданной метке раздела.

Распакуем архив на раздел ROOTFS:

```
$ sudo tar -xJpf /путь/до/rootfs.tar.xz -C /путь/до_раздела/ROOTFS/
```

Готово! Можно пробовать вставлять sd-карту в разъем одноплатника, перед извлечением кардридера не забудьте про вызов команды **sync**, подключаться к пирам UART с помощью переходника и наблюдать за процессом запуска.

6.5 Контрольные вопросы

1. Почему для загрузочной SD-карты необходимо создавать два раздела?
2. Зачем загрузчик U-Boot записывается на SD-карту со смещением 8 килобайт (bs=1k seek=8)?
3. Какая файловая система используется для раздела BOOT и почему?
4. Какие компоненты необходимо скопировать на SD-карту для успешной загрузки системы?

6.6 Задание

Ознакомившись с подготовительным материалом к лабораторным №5 и №6 решить следующие подзадачи:

- Создать полноценный образ Alt Linux из подготовленных в предыдущей лабораторной работе компонентов
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продemonстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

7 Лабораторная №7. Добавление поддержки SPI интерфейса

7.1 Цель работы

Изучить принципы работы SPI-интерфейса в одноплатных компьютерах под управлением Linux, освоить методы конфигурирования ядра и модификации Device Tree для добавления поддержки нового аппаратного интерфейса, а также получить практические навыки взаимодействия с SPI-устройствами из пользовательского пространства.

7.2 Подготовительный материал

7.2.1 Краткая теория о самом интерфейсе

SPI (Serial Peripheral Interface) — это интерфейс для связи с периферийными устройствами, такими как датчики, преобразователи и память. На рисунке ниже показана топология SPI с несколькими ведомыми устройствами:

Архитектура интерфейса строится по принципу «ведущий-ведомый» (master-slave), где ведущий формирует тактовый сигнал синхронизации (SCK). Передача данных осуществляется по двум линиям: MOSI (Master Out Slave In) — данные от ведущего к ведомому, и MISO (Master In Slave Out) — данные от ведомого к ведущему. Это позволяет реализовать полнодуплексный обмен информацией. Важной особенностью интерфейса является то, что при каждой записи команды или данных ведомое устройство всегда возвращает ответ той же длины, даже если этот ответ не содержит полезной информации.

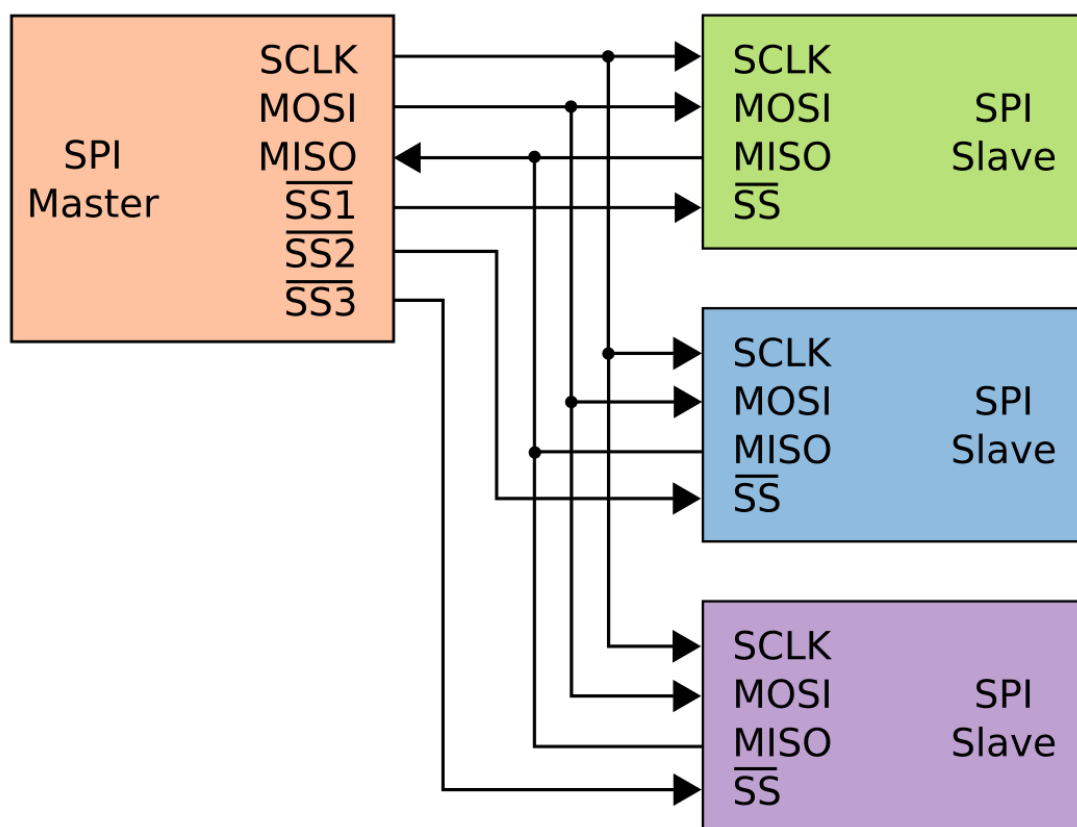


Рис. 7.1 - alt text

Классическая схема подключения, приведенная на рисунке выше, является параллельной: все ведомые устройства совместно используют линии данных (MOSI, MISO) и тактирования (SCK). Единственным исключением является сигнал выбора ведомого ($\sim CS$), который для каждого устройства должен быть индивидуальным. На рисунке эти сигналы обозначены как SSx. Для их формирования можно использовать как специализированные выводы SPI-контроллера, так и универсальные GPIO-пины.

В случае с платой Lichee RV Dock базово предоставляется возможность подключить только одно SPI-устройство, что упрощает схему (см. рисунок ниже).

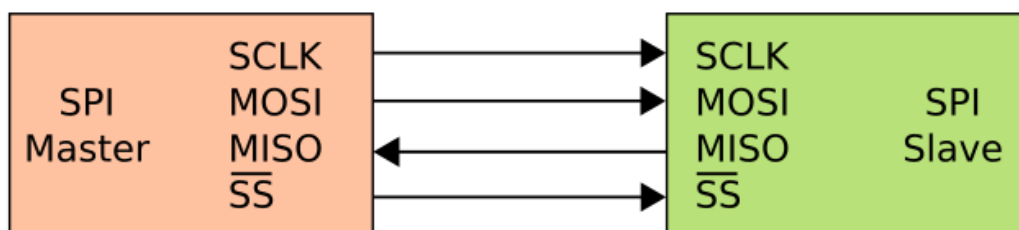


Рис. 7.2 – alt text

Интерфейс SPI применяется преимущественно во встроенных системах и электронных устройствах для короткой дистанции, обеспечивая высокоскоростной синхронный обмен данными между микроконтроллером (мастером) и периферийными микросхемами (ведомыми). Основные цели его использования — подключение и управление разнообразной периферией, такой как датчики (температуры, давления, акселерометры), карты памяти (SD, MMC), ЦАП/АЦП, сдвиговые регистры, драйверы дисплеев и тд.

7.3 Постановка задачи

Основной целью лабораторной работы является обеспечение возможности взаимодействия с SPI-интерфейсом микроконтроллера Allwinner D1 из пользовательского пространства операционной системы Linux.

Speed Lichee RV DOCK pinout diagram

The diagram illustrates the pinout for the Speed Lichee RV DOCK, centered around a 40-pin GPIO header. The pins are organized into four columns, each with a specific function group.

- Multiple Functions (V_{DD}=3.3V):** This column includes pins for PWM4, PWM3, PWM2, UART3_TX, UART3_RX, DMIC_D2, SPI1_WP, SPI1_MISO, SPI1_MOSI, SPI1_CS, DMIC_D0, DMIC_D1, LVDS0_V3P, LVDS0_CK, LVDS0_V2P, LVDS0_V1P, LVDS0_V0P, and DMIC_D1.
- Default Functions (V_{DD}=3.3V):** This column includes pins for TP_TWI*, TP_INT, RGB_R2, DSI_RST, RGB_RST, RGB_R5, RGB_R3, RGB_R1, RGB_R0, RGB_G4, RGB_DE, DSI_BL, RGB_CLK, DSI_D3P, DSI_D2P, DSI_CK, DSI_D1P, DSI_D0P, RGB_INT0, and GND.
- 40-pin GPIO header:** The central header with 40 pins, labeled from PB1 to PE16, with GND at the bottom.
- Default Functions (V_{DD}=3.3V):** This column includes pins for 3V3, 5V, 5V, GND, PB8, PB9, PG15, GND, PD16, PD22, GND, PD11, VSYNC, HSYNC, PD9, PD7, PD5, PD3, PD1, RGB_B1, RGB_INT2, and RGB_INT2.
- Multiple Functions (V_{DD}=3.3V):** This column includes pins for UART0_TX, UART0_RX, TP_RST, RGB_R4, RGB_BL, RGB_G5, RGB_G1, RGB_G3, RGB_G1, RGB_B3, RGB_B1, RGB_INT2, and RGB_INT2.

Legend:

- Power: Red
- GND: Dark Blue
- GPIO: Grey
- TWI: Cyan
- MIPI_DSI: Orange
- RGB_LCD: Light Blue
- Touch Panel: White
- UART: Yellow
- PWM: Green
- SPI: Light Green
- DMIC: Dark Green
- LVDS_DSI: Light Green

SPI-интерфейс выведен на GPIO-разъем в качестве альтернативной функции. Это означает, что по умолчанию соответствующие выводы процессора сконфигурированы для работы с RGB-экраном. Следовательно, первая задача формулируется следующим образом: сконфигурировать выводы микроконтроллера для работы в режиме SPI-интерфейса.

43

7.3.1 Необходимые шаги

Для успешного решения поставленных задач необходимо выполнить следующие действия:

- Добавление и активация интерфейса в Device Tree
- Добавление драйвера поддержки интерфейса SPI в ядро
- Добавление сопутствующих драйверов

7.3.2 Добавление и активация интерфейса в Device Tree

Для понимания процесса формирования бинарных .dtb файлов из исходных текстов дерева устройств необходимо рассмотреть структуру соответствующих файлов для платы Lichee RV Dock.

В директории исходных кодов ядра `arch/riscv/boot/dts/allwinner/` расположены исходные тексты дерева устройств для одноплатных компьютеров на базе чипа Allwinner D1.

При анализе файла `sun20i-d1-lichee-rv-dock.dts` можно заметить включение в него аналогичного файла `sun20i-d1-lichee-rv.dts` для базовой версии одноплатного компьютера. В свою очередь, данный файл содержит включение файлов с более общими параметрами. В результате последовательного включения достигается файл `sunxi-d1s-t113.dtsi`, в котором объявлены все интерфейсы и устройства, поддерживаемые микроконтроллером D1. Файлы с расширением .dtsi выполняют функцию заголовочных файлов для исходных файлов формата .dts, где суффикс *i* обозначает include.

В заголовочном файле содержатся следующие объявления, относящиеся к SPI-интерфейсу:

Объявление выводов микроконтроллера, поддерживающих работу двух SPI-интерфейсов (spi0 и spi1). Соответствие объявлений аппаратной конфигурации можно проверить по схеме одноплатного компьютера:

```
/omit-if-no-ref/
spi0_pins: spi0-pins {
    pins = "PC2", "PC3", "PC4", "PC5";
    function = "spi0";
};
```

```

/omit-if-no-ref/
spi1_pb_pins: spi1-pb-pins {
    pins = "PB0", "PB8", "PB9", "PB10", "PB11", "PB12";
    function = "spi1";
};

/omit-if-no-ref/
spi1_pd_pins: spi1-pd-pins {
    pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
    function = "spi1";
};

```

Важно: директива `/omit-if-no-ref/` предотвращает включение в результирующий .dtb файл объектов, которые не используются после объявления.

Объявление SPI интерфейсов, в которых указано с какими аппаратными прерываниями, интерфейсами и железом должен пользоваться интерфейс:

```

spi0: spi@4025000 {
    compatible = "allwinner,sun20i-d1-spi",
                 "allwinner,sun50i-r329-spi";
    reg = <0x4025000 0x1000>;
    interrupts = <31 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_SPI0>, <&ccu CLK_SPI0>;
    clock-names = "ahb", "mod";
    resets = <&ccu RST_BUS_SPI0>;
    dmas = <&dma 22>, <&dma 22>;
    dma-names = "rx", "tx";
    num-cs = <1>;
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
};

spi1: spi@4026000 {
    compatible = "allwinner,sun20i-d1-spi-dbi",
                 "allwinner,sun50i-r329-spi-dbi",
                 "allwinner,sun50i-r329-spi";
    reg = <0x4026000 0x1000>;
};

```

```

interrupts = <32 IRQ_TYPE_LEVEL_HIGH>;
clocks = <&ccu CLK_BUS_SPI1>, <&ccu CLK_SPI1>;
clock-names = "ahb", "mod";
resets = <&ccu RST_BUS_SPI1>;
dmas = <&dma 23>, <&dma 23>;
dma-names = "rx", "tx";
num-cs = <1>;
status = "disabled";
#address-cells = <1>;
#size-cells = <0>;
};

```

Параметр `compatible` содержит идентификаторы драйверов, поддерживающих работу с соответствующим устройством. Первым указывается предпочтительный драйвер для инициализации устройства. При загрузке ядра драйверы анализируют значение данного поля у активных устройств и при обнаружении поддерживаемого устройства вызывают функцию `probe` для его инициализации в операционной системе.

Обратите внимание, что статус обоих устройств имеет значение `"disabled"`. Для активации интерфейса необходимо изменить данный статус в исходном файле платы, добавив следующую конфигурацию:

```

&spil {
    pinctrl-0 = <&spil_pd_pins>;
    pinctrl-names = "default";
    status = "okay";
};

```

Данная запись активирует интерфейс и определяет используемые выводы для обмена данными.

Для обеспечения доступа к SPI-интерфейсу из пользовательского пространства необходим драйвер `SPIDEV`, поддержку которого требуется включить в конфигурацию ядра (параметр `SPI_SPIDEV`). Согласно документации, для автоматического подключения драйвера устройство должно присутствовать в одной из таблиц устройств, определенных в исходном коде драйвера. Существует два подхода к решению данной задачи:

- Добавление идентификатора устройства в одну из таблиц и компиляция модифицированного драйвера

- Использование существующего идентификатора из таблиц при объявлении устройства в Device Tree

В данной работе предлагается использовать второй подход:

```
&spi1{
    pinctrl-0 = <&spi1_pd_pins>;
    pinctrl-names = "default";
    status = "okay";
    spidev0: spidev@0 {
        compatible = "rohm,dh2228fv";
        spi-max-frequency = <100000000>;
        reg = <0x00>;
    };
};
```

На этом модификация Device Tree завершена.

7.3.3 Добавление драйвера поддержки интерфейса SPI в ядро

В качестве базовой конфигурации используется конфигурационный файл, полученный модификацией defconfig для Lichee RV Dock в лабораторной №4.

Базовая поддержка SPI-интерфейса (параметры SPI и SPI_MASTER) скорее всего уже присутствует в данной конфигурации. Однако для работы с конкретным SoC требуется соответствующий драйвер.

Несмотря на наличие в поле `compatible` заголовочного файла Device Tree соответствующих идентификаторов драйверов, поддержка семейства `sun20i` в стандартном драйвере отсутствовала. Решение данной проблемы было реализовано через патч, добавляющий поддержку SPI для ряда семейств процессоров Allwinner (описание изменений). Модификации затронули драйвер `spi-sun6i.c`, изначально предназначенный для устройств на архитектуре ARM. Ввиду схожей реализации SPI-интерфейса в драйвер была добавлена поддержка дополнительных семейств, включая `sun20i`.

Для включения поддержки необходимо активировать в конфигурации ядра параметр `SPI_SUN6I`. **(необходимо будет это сделать самостоятельно)**

7.3.4 Добавление сопутствующих драйверов

В процессе тестирования драйвера SPI_SUN6I было обнаружено, что инициализация устройства не происходит из-за невозможности доступа к DMA-каналам. Анализ показал необходимость наличия драйвера DMA для соответствующего семейства микроконтроллеров. Добавление драйвера DMA_SUN6I позволило успешно завершить инициализацию SPI-интерфейса. **(необходимо будет это сделать самостоятельно)**

7.3.5 Результат работы

После выполнения всех шагов в каталоге */dev/* появляется символическое устройство *spidev*. Для проверки работоспособности интерфейса можно выполнить тестирование путем замыкания выводов MOSI и MISO и запуска скрипта на языке Python:

```
import spidev
import time

def spi_sequential_test(bus=0, device=0, speed=1000000):

    spi = spidev.SpiDev()
    spi.open(bus, device)
    spi.max_speed_hz = speed
    spi.mode = 0
    spi.lsbfirst = False

    print("SPI последовательный тест запущен...")
    print("Отправляю данные: 0, 1, 2, 3, ...")
    print("-" * 40)

    for i in range(256): # От 0 до 255
        # Отправляем одно число и получаем ответ
        data_to_send = [i]
        response = spi.xfer2(data_to_send)

        print(f"Отправлено: {data_to_send[0]:3d} (0x{data_to_send[0]:02X}) | "
              f"Получено: {response[0]:3d} (0x{response[0]:02X}) | "
              f"Бинарно: {response[0]:08b}")
```



```

        time.sleep(0.01) # Небольшая пауза для анализатора

    spi.close()

# Запуск теста
if __name__ == "__main__":
    # Настройте под вашу плату
    spi_sequential_test(bus=0, device=0, speed=1000000)

```

Результат:

```

[root@localhost my_spi_device]# python3 spi.py
SPI последовательный тест запущен...
Отправляю данные: 0, 1, 2, 3, ...
-----
Отправлено:  0 (0x00) | Получено:  0 (0x00) | Бинарно: 00000000
Отправлено:  1 (0x01) | Получено:  1 (0x01) | Бинарно: 00000001
Отправлено:  2 (0x02) | Получено:  2 (0x02) | Бинарно: 00000010
Отправлено:  3 (0x03) | Получено:  3 (0x03) | Бинарно: 00000011
Отправлено:  4 (0x04) | Получено:  4 (0x04) | Бинарно: 00000100
Отправлено:  5 (0x05) | Получено:  5 (0x05) | Бинарно: 00000101
Отправлено:  6 (0x06) | Получено:  6 (0x06) | Бинарно: 00000110
Отправлено:  7 (0x07) | Получено:  7 (0x07) | Бинарно: 00000111
Отправлено:  8 (0x08) | Получено:  8 (0x08) | Бинарно: 00001000
Отправлено:  9 (0x09) | Получено:  9 (0x09) | Бинарно: 00001001
Отправлено: 10 (0x0A) | Получено: 10 (0x0A) | Бинарно: 00001010

```

Рис. 7.4 – Результат тестирования SPI замыканием MOSI/MISO

7.4 Контрольные вопросы

1. Какие четыре сигнала используются в интерфейсе SPI и за что отвечает каждый?
2. Чем архитектура «ведущий-ведомый» в SPI отличается от шинной архитектуры I2C?
3. Почему в Device Tree статус интерфейса (status) по умолчанию имеет значение "disabled"?
4. Зачем нужен драйвер SPIDEV и какое символьное устройство появляется в /dev/ после его активации?

5. Какой идентификатор в поле `compatible` используется для привязки драйвера `spidev` к устройству в Device Tree?
6. Почему для работы SPI-интерфейса на Allwinner D1 требуется драйвер `DMA_SUN6I`?

7.5 Задание

Ознакомившись с подготовительным материалом к лабораторным №5, №6 и №7 решить следующие подзадачи:

- Проанализировать файлы Device Tree платы Lichee RV Dock и определить необходимые модификации для активации SPI-интерфейса
- Внести изменения в исходные файлы дерева устройств для активации SPI1 на выводах PD10-PD15
- Добавить в конфигурацию ядра поддержку:
 - SPI-интерфейса (убедиться в готовности параметров `SPI` и `SPI_MASTER`)
 - Драйвера `SPI_SUN6I` для работы с SoC Allwinner D1
 - Драйвера `DMA_SUN6I` для обеспечения доступа к DMA-каналам
 - Драйвера `SPIDEV` для работы с SPI из пользовательского пространства
- Собрать ядро с модифицированной конфигурацией и обновленными файлами дерева устройств
- Выполнить загрузку системы с новым ядром и проверить наличие устройства `/dev/spidev*`
- Провести тестирование работоспособности SPI-интерфейса методом обратной связи (замыкание MOSI и MISO)
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач `.doc` и **выслать на почту преподавателя до обозначенного срока**

8 Лабораторная №8. Добавление поддержки I2C интерфейса

8.1 Цель работы

Изучить принципы работы I2C-интерфейса в одноплатных компьютерах под управлением Linux, освоить методы конфигурирования ядра и модификации Device Tree для добавления поддержки нового аппаратного интерфейса, а также получить практические навыки взаимодействия с I2C-устройствами из пользовательского пространства.

8.2 Подготовительный материал

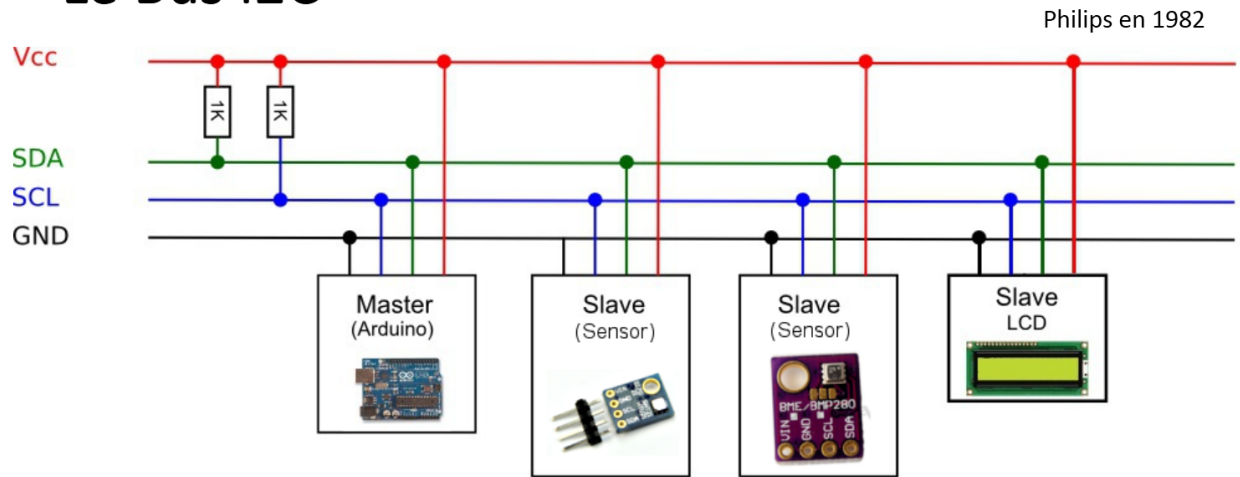
8.2.1 Краткая теория о самом интерфейсе

I2C является шиной, т.е. к одним и тем же выводам может быть подключено несколько устройств. По стандарту на шине есть только одно ведущее устройство - Master, остальные - ведомые, Slave. Ведущее выбирает, с каким из ведомых взаимодействовать, может отправлять и читать с него данные. Ведущим обычно является основной МК в схеме, а остальные - различные цифровые микросхемы, датчики или вспомогательные МК. На рисунке ниже показана многоточечная топология шины I2C:

8.3 Постановка задачи

Основной целью лабораторной работы является обеспечение возможности взаимодействия с I2C-интерфейсом микроконтроллера Allwinner D1 из пользовательского пространства операционной системы Linux.

Le Bus I2C



- 2 fils de connexion (SCL : horloge, SDA: donnée)
- 127 adresses ou esclave possibles
- Vitesse 100kb/s à 400kb/s (Arduino)
- Transmission synchrone

14/21

Рис. 8.1 – alt text

Для достижения данной цели необходимо решить ряд задач, возникающих при анализе аппаратной конфигурации исследуемой платы. При обращении к распиновке (схема или пин-аут) платы Lichee RV Dock можно не обнаружить записей об I2C. На самом деле **промышленное название I2C - TWI**. Поэтому нас будут интересовать пины TWI2_SDA и TWI2_SCL.

I2C использует 2 пина для подключения:

- SDA (Serial Data) - линия данных, передача в обе стороны. На модуле может быть подписан как SDA, D
- SCL (Serial Clock) - линия синхронизации, ей управляет мастер. На модуле может быть подписан как SCL, C, SCK

8.3.1 Необходимые шаги

Для успешного решения поставленных задач необходимо выполнить следующие действия:

- Добавление и активация интерфейса в Device Tree
- Добавление драйвера поддержки интерфейса I2C в ядро

**Sipeed Lichee RV DOCK
pinout diagram**

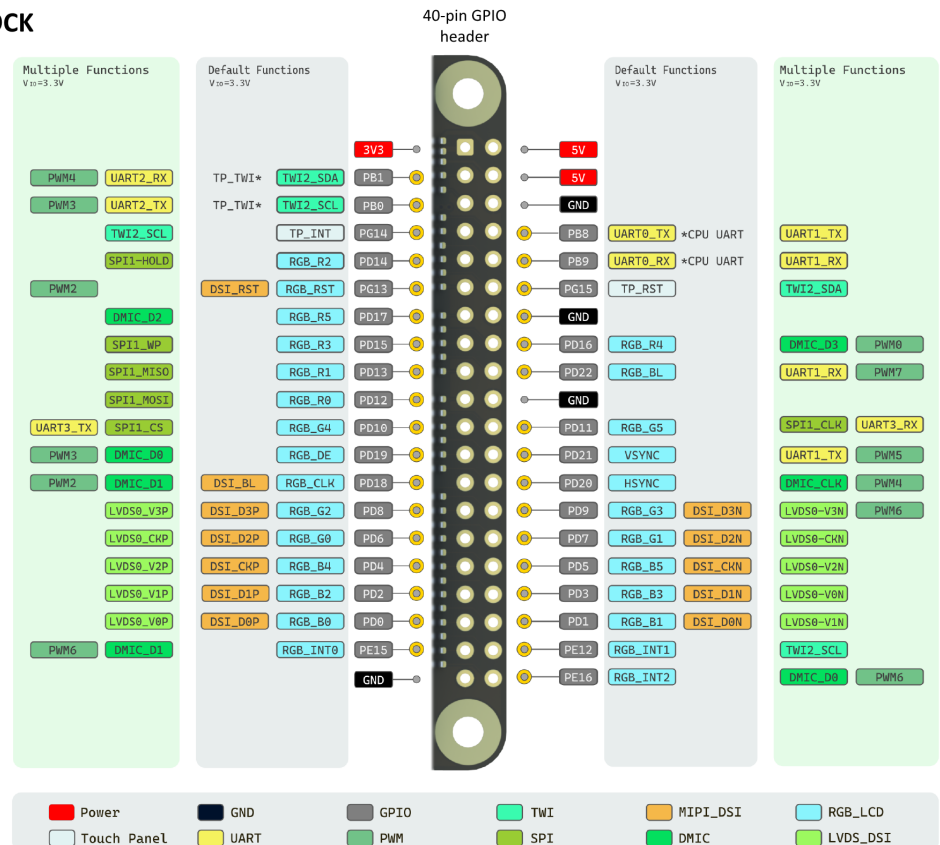


Рис. 8.2 - Распиновка Lichee RV Dock — пины TWI2_SDA и TWI2_SCL

8.3.2 Добавление и активация интерфейса в Device Tree

Необходимо по аналогии с задачей из лабораторной №7 в исходный файл платы добавить(или убедиться в наличии) переобъявление некоторых полей для i2c-интерфейса:

```
&i2c2 {
    pinctrl-0 = <&i2c2_pepg_pins>;
    pinctrl-names = "default";
    #compatible = "marvell,mv64xxx-i2c";
    status = "okay";
};
```

В заголовочный файл *sun20i-d1.dtsi* необходимо добавить объявление пинов SDA и SCL:

```
/omit-if-no-ref/
i2c2_pepg_pins: i2c2-pepg-pins {
    pins = НОМЕРА ПИНОВ ДЛЯ ИНТЕРФЕЙСА;
```

```
        function = "i2c2";  
};
```

8.3.3 Добавление драйвера поддержки интерфейса I2C в ядро

Необходимо по аналогии с задачей из лабораторной №7 добавить в конфигурацию ядра поддержку следующих параметров:

- I2C
- I2C_MUX
- I2C_MV64XXX
- I2C_CHARDEV

8.3.4 Добавление

Для простой работы с I2C-экраном и запуска тестового скрипта необходимо поставить пакет для нативной работы с такими экранами.

```
# apt-get install python3-module-luma-oled
```

8.3.5 Результат работы

После выполнения всех шагов в каталоге */dev/* появляется символическое устройство *i2c-0*. Для проверки работоспособности интерфейса можно выполнить тестирование путем запуска программы вывода собственного имени и фамилии студента на экран SSD-1306:

```
#!/usr/bin/env python3  
  
import sys  
import time  
from luma.core.interface.serial import i2c  
from luma.oled.device import ssd1306  
from luma.core.render import canvas  
from PIL import ImageFont, ImageDraw  
  
def display_large_centered(device, first_name, last_name):
```

```

"""Выводит имя и фамилию крупным шрифтом по центру"""

# Преобразуем в нижний регистр
first_name_lower = first_name.lower()
last_name_lower = last_name.lower()

# Пытаемся загрузить крупный шрифт
try:
    # Для Raspberry Pi с установленными шрифтами
    font_large = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/Deja
    font_small = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/Deja
except:
    # Если шрифт не найден, используем шрифт по умолчанию
    font_large = ImageFont.load_default()
    font_small = ImageFont.load_default()

with canvas(device) as draw:
    # Для крупного шрифта используем две строки
    # Получаем размеры текста
    bbox_name = draw.textbbox((0, 0), first_name_lower, font=font_large)
    bbox_surname = draw.textbbox((0, 0), last_name_lower, font=font_large)

    name_width = bbox_name[2] - bbox_name[0]
    surname_width = bbox_surname[2] - bbox_surname[0]

    # Центрируем каждую строку
    x_name = (128 - name_width) // 2
    x_surname = (128 - surname_width) // 2

    # Выводим по центру
    draw.text((x_name, 15), first_name_lower, fill="white", font=font_large)
    draw.text((x_surname, 38), last_name_lower, fill="white", font=font_large)

def main():
    if len(sys.argv) != 3:
        print("Ошибка: нужно указать имя и фамилию как параметры запуска")
        print(f"Пример: {sys.argv[0]} Иван Петров")
        sys.exit(1)

    serial = i2c(port=0, address=0x3C)

```

```

device = ssd1306(serial)
device.clear()

# Выводим имя и фамилию крупным шрифтом по центру
display_large_centered(device, sys.argv[1], sys.argv[2])

print(f"Выведено на экран: {sys.argv[1].lower()} {sys.argv[2].lower()}")

# Держим экран включенным 10 секунд
time.sleep(10)

if __name__ == "__main__":
    main()

```

Для работы с I2C-экраном необходимо подключить **пин VCC на нем с пином питания(5V) на одноплатнике, пины SDA, SCL, GND соответственно.**

Результатом зафиксированным в отчете должен стать вывод на экранчик фамилии и имени студента.

8.4 Контрольные вопросы

1. Почему интерфейс I2C на Lichee RV Dock обозначен как TWI? В чём разница?
2. Как организована адресация устройств на шине I2C?
3. Чем отличаются линии SDA и SCL по своему назначению в I2C?
4. Какие параметры ядра необходимо включить для поддержки I2C на Allwinner D1?
5. Для чего нужен драйвер I2C_CHARDEV и какое устройство появляется в /dev/?
6. Как определить I2C-адрес подключённого устройства, если он неизвестен?

8.5 Задание

Ознакомившись с подготовительным материалом к лабораторным №5, №6, №7 и данной лабораторной №8 решить следующие подзадачи:

- Проанализировать файлы Device Tree платы Lichee RV Dock и внести необходимые модификации для активации I2C-интерфейса пинах из банков PE и PG
- Добавить в конфигурацию ядра поддержку I2C-интерфейса путем включения перечисленных в материалах лабораторной параметров в конфигурацию ядра
- Собрать ядро с модифицированной конфигурацией и обновленными файлами дерева устройств
- Выполнить загрузку системы с новым ядром и проверить наличие устройства /dev/i2c-0
- Провести тестирование работоспособности I2C-интерфейса путем запуска программы вывода собственного имени и фамилии студента на экран SSD-1306
- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчет о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

9 Лабораторная №9. Работа с логическими анализаторами

9.1 Цель работы

Изучить принципы работы логического анализатора, освоить методы использования библиотеки `libgpiod2` для управления выводами GPIO, получить практические навыки анализа цифровых сигналов и протоколов с использованием программного обеспечения Sigrok/PulseView.

9.2 Подготовительный материал

9.2.1 Логический анализатор

При работе с различными электронными устройствами для анализа и понимания процессов на физическом уровне часто требуется использование специализированных измерительных приборов. Самым точным инструментом для анализа цифровых сигналов является осциллограф. Однако в случаях, когда высокая точность измерений не требуется, но необходима декодировка сигнала или длительная запись измерений, эффективным решением становится логический анализатор.

Логический анализатор — это инструмент для отладки цифровых протоколов, анализа временных характеристик и обнаружения аппаратных проблем. Прибор позволяет одновременно отслеживать состояние нескольких цифровых линий (каналов) и записывать изменения сигналов во времени.

В операционной системе ALT Linux доступны пакеты проекта Sigrok:

- `sigrok-cli` — утилита командной строки

- pulseview — графический интерфейс

Sigrok — программный проект, обеспечивающий поддержку различных анализаторов сигналов. **Pulseview** — графическая оболочка для визуализации и декодирования данных.

9.2.2 Установка программного обеспечения

Для работы с логическим анализатором необходимо установить следующие пакеты:

```
# apt-get install sigrok-cli sigrok-firmware-fx2lafw pulseview
```

9.2.3 Пример использования логического анализатора

GPIO (General Purpose Input/Output) — выводы общего назначения, которые могут работать как входы или выходы. Они позволяют микроконтроллеру взаимодействовать с внешними устройствами: читать сигналы от датчиков, управлять светодиодами, реле, двигателями и другими периферийными устройствами. Выводы GPIO являются основным интерфейсом для связи одноплатного компьютера с внешним миром.

Типичная задача для логического анализатора — измерение параметров цифрового сигнала на выводе GPIO. Например, на определённом пине гребёнки платы формируется сигнал с длительностью импульса 10 мс.

Для подключения анализатора необходимо соединить:

- Вывод GND анализатора с соответствующим пином земли на плате
- Канал CH1-CH8 с исследуемым пином

Запуск PulseView и выбор устройства:

```
$ pulseview
```

В окне выбора устройства выбрать анализатор fx2lafw и нажать Start. Результатом является график изменения сигналов по каналам, позволяющий измерить временные параметры импульсов — длительность высокого и низкого уровня, период и частоту сигнала.

9.2.4 Анализ протокола UART

Практическое применение логического анализатора — анализ работы последовательного интерфейса UART. На плате Lichee RV Dock при загрузке в UART-консоль выводится диагностическая информация.

Подключение анализатора:

- Вывод GND анализатора — к пину GND платы
- Канал CH0 — к пину TX платы

Настройка PulseView для анализа UART:

1. Удалить лишние каналы, оставить первый
2. Установить частоту дискретизации 1 MHz
3. Добавить декодер протокола: кнопка “Add protocol decoder” → UART

Параметры декодирования UART:

Baud rate: 115200
Data bits: 8
Parity: 0
Stop bits: 1
Bit order: lsb-first
Data format: ascii
Invert RX: no
Invert TX: no
Sample point (%): 50

Применение настроек позволяет декодировать передаваемые данные и отобразить их в виде символов ASCII.

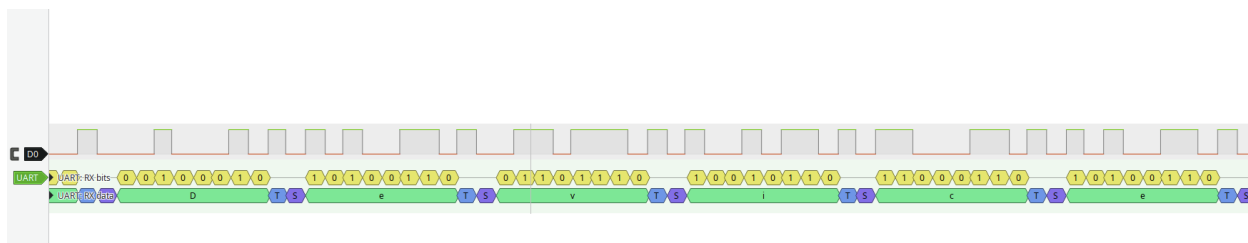


Рис. 9.1 – Показания анализатора

9.2.5 Библиотека libgpiod2

Библиотека libgpiod2 предоставляет удобный интерфейс для управления выводами общего назначения (GPIO) из пользовательского пространства операционной системы Linux. Библиотека является частью проекта GNU и распространяется под лицензией LGPL-2.1+.

Основные концепции:

gpiochip — символьное устройство, представляющее один или несколько выводов GPIO. В системе может быть несколько gpiochip-устройств, каждое из которых имеет уникальное имя и диапазон номеров линий.

Линия GPIO — отдельный вывод, который может быть настроен как вход или выход. Каждая линия идентифицируется номером в пределах gpiochip.

Потребитель (consumer) — имя процесса, использующего линию GPIO. Используется для идентификации владельца в системе.

API библиотеки libgpiod2:

Основные структуры и функции:

```
#include <gpiod.h>
```

```
// Открытие gpiochip
```

```
struct gpiod_chip *gpiod_chip_open(const char *path);
```

```
// Создание настроек для линии
```

```
struct gpiod_line_settings *gpiod_line_settings_new();
```

```
// Установка направления линии (вход/выход)
```

```
void gpiod_line_settings_set_direction(struct gpiod_line_settings *settings,  
                                     enum gpiod_line_direction direction);
```

```
// GPIOD_LINE_DIRECTION_INPUT или GPIOD_LINE_DIRECTION_OUTPUT
```

```
// Установка начального значения для выхода
```

```
void gpiod_line_settings_set_output_value(struct gpiod_line_settings *settings,  
                                          unsigned int value);
```

```
// Освобождение настроек линии
```

```
void gpiod_line_settings_free(struct gpiod_line_settings *settings);
```

```

// Создание конфигурации для нескольких линий
struct gpiod_line_config *gpiod_line_config_new();

// Добавление настроек для конкретных линий
int gpiod_line_config_add_line_settings(struct gpiod_line_config *config,
                                         const unsigned int *offsets,
                                         size_t num_offsets,
                                         struct gpiod_line_settings *setting);

// Освобождение конфигурации линий
void gpiod_line_config_free(struct gpiod_line_config *config);

// Создание конфигурации запроса
struct gpiod_request_config *gpiod_request_config_new();

// Установка имени потребителя (процесса)
void gpiod_request_config_set_consumer(struct gpiod_request_config *config,
                                         const char *consumer);

// Освобождение конфигурации запроса
void gpiod_request_config_free(struct gpiod_request_config *config);

// Запрос линий у чипа
struct gpiod_line_request *gpiod_chip_request_lines(struct gpiod_chip *chip,
                                                      struct gpiod_request_config *config,
                                                      struct gpiod_line_config *line_config);

// Установка значения на выводе
int gpiod_line_request_set_value(struct gpiod_line_request *request,
                                   unsigned int offset, int value);

// Освобождение запроса линий
void gpiod_line_request_release(struct gpiod_line_request *request);

// Закрытие чипа
void gpiod_chip_close(struct gpiod_chip *chip);

```

Пример использования (генератор меандра):

Простой пример генерации меандра на выводе GPIO:

```
#include <gpiod.h>
```

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

#define GPIO_CHIP    "/dev/gpiochip0"
#define GPIO_OFFSET  144  // Line 144 - PE16

static volatile int keep_running = 1;

static void signal_handler(int signum)
{
    (void)signum;
    keep_running = 0;
}

int main() {
    struct gpiod_chip *chip;
    struct gpiod_line_request *req;
    struct gpiod_line_settings *settings;
    struct gpiod_line_config *line_cfg;
    struct gpiod_request_config *req_cfg;
    unsigned int offset = GPIO_OFFSET;

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    chip = gpiod_chip_open(GPIO_CHIP);
    if (!chip) {
        perror("gpiod_chip_open");
        return 1;
    }

    settings = gpiod_line_settings_new();
    gpiod_line_settings_set_direction(settings, GPIOD_LINE_DIRECTION_OUTPUT);
    gpiod_line_settings_set_output_value(settings, 0);

    line_cfg = gpiod_line_config_new();
    gpiod_line_config_add_line_settings(line_cfg, &offset, 1, settings);

```

```

req_cfg = gpiod_request_config_new();
gpiod_request_config_set_consumer(req_cfg, "wavegen");

req = gpiod_chip_request_lines(chip, req_cfg, line_cfg);
if (!req) {
    perror("gpiod_chip_request_lines");
    gpiod_chip_close(chip);
    return 1;
}

printf("Generating square wave on GPIO line %u (press Ctrl+C to stop)\n",
line);

while (keep_running) {

    // Генерация меандра должна быть здесь

}

printf("\nCleaning up...\n");

gpiod_line_request_release(req);
gpiod_chip_close(chip);
gpiod_line_settings_free(settings);
gpiod_line_config_free(line_cfg);
gpiod_request_config_free(req_cfg);

printf("Done\n");
return 0;
}

```

Установка библиотеки:

В ALT Linux библиотека доступна в пакете:

```
# apt-get install libgpiod2 libgpiod-devel
```

Пакет `libgpiod-devel` содержит заголовочные файлы для компиляции приложений.

Компиляция программы:

Компиляция с использованием `libgpiod2`:


```
$ gcc blink.c -o blink -lgpiod
```

Получение информации о доступных выводах:

Просмотр информация о gpiochip и линиях:

```
$ gpioinfo                # вся информация о GPIO
$ gpioget -c 0 144        # состояние линии 144 на чипе 0
$ gpiowrite gpiochip0 144=1 # установить состояние линии 144 на чипе 0 в "1"
$ gpiomon -c 0 144        # отслеживать прерывания на линии 144 на чипе 0
```

9.2.6 Задача. Генератор меандра на GPIO

Необходимо, пользуясь примером приведенным ранее создать генератор, реализованный как консольное приложение на языке Си с использованием библиотеки `libgpiod2`. Параметры длительности должны передаваться через аргументы командной строки в микросекундах. Генерация сигнала должна происходить в непрерывном режиме до получения сигнала прерывания (Ctrl+C), после чего ресурсы (линия GPIO) корректно освобождаются.

Вывод GPIO для генерации — PE16 (линия 144 на `gpiochip0`).

Пример запуска генератора с периодом 500 мкс (250 мкс высокий уровень, 250 мкс низкий уровень):

```
$ ./generator 250 250
```

Анализ работы генератора:

Для анализа характеристик сигнала используется логический анализатор в связке с программой PulseView:

1. Подключить канал CH0 анализатора к выводу GPIO с генератором
2. Запустить генератор
3. Запустить захват сигнала в PulseView
4. Измерить параметры сигнала: длительность импульсов высокого и низкого уровня, частоту

9.3 Контрольные вопросы

1. Для чего нужен логический анализатор при отладке цифровых сигналов?
2. Какие параметры необходимо настроить в PulseView для декодирования UART-сигнала?
3. Что такое GPIO-линия в терминах libgpiod2 и как к ней обратиться из программы?
4. Чем управление GPIO через libgpiod2 принципиально отличается от прямой записи в регистры микроконтроллера (как в Arduino)?

9.4 Задание

Ознакомившись с подготовительным материалом решить следующие подзадачи:

9.4.1 Работа с логическим анализатором

- Установить пакеты sigrok-cli и pulseview
- Подключить логический анализатор к персональному компьютеру
- Запустить pulseview и убедиться в обнаружении анализатора
- Выполнить измерение параметров тестового сигнала (импульс 10 мс)
- Провести анализ вывода данных в UART-консоль одноплатника, подключив анализатор к пину TX

9.4.2 Работа с libgpiod2

- Установить пакет libgpiod-devel
- Изучить распиновку вывода PE16 на плате Lichee RV Dock
- Собрать пример генератора на основе примера из подготовительного материала
- Запустить генератор и проверить его работу

9.4.3 Создание генератора меандра

- Разработать консольное приложение генератора меандра с использованием libgpiod2
- Обеспечить приём параметров длительности через аргументы командной строки
- Реализовать корректное освобождение ресурсов при прерывании (Ctrl+C)
- Собрать и запустить генератор на выводе PE16

9.4.4 Анализ работы генератора

- Подключить логический анализатор к выводу GPIO с генератором
- Выполнить захват сигнала в pulseview
- Измерить длительность импульсов высокого и низкого уровня
- Сохранить результаты измерений в файл

9.4.5 Демонстрация и отчёт

- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчёт о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

10 Лабораторная №10. Основы программирования микроконтроллеров Arduino. Часть 1.

10.1 Цель работы

Освоить базовые принципы программирования микроконтроллеров на примере платформы Arduino. Понять ключевые отличия программирования “чистого железа” от разработки под операционную систему. Научиться создавать простые программы для управления светодиодами и обработки внешних прерываний, а также интегрировать Arduino с одноплатником для совместной работы.

10.2 Подготовительный материал

10.2.1 Что такое Arduino?

Arduino — это открытая платформа для прототипирования электронных устройств, основанная на простой в использовании аппаратной и программной среде. По сути, это микроконтроллерный комплект, который позволяет быстро создавать электронные проекты без глубоких знаний в электронике.

Ключевые особенности Arduino: - Микроконтроллер — “мозг” платы (чаще всего ATmega328P) - Программная оболочка — упрощённая среда разработки (Arduino IDE) - Стандартизированные разъёмы — для подключения периферии - Открытая архитектура — возможность модификации и клонирования

10.2.2 Программирование “чистого железа” vs разработка под ОС

Ключевые особенности Arduino:

Программирование микроконтроллеров, таких как Arduino, принципиально отличается от разработки под операционные системы. В Arduino отсутствует операционная система — программа работает напрямую с аппаратным обеспечением, что обеспечивает прямой доступ к регистрам периферийных устройств. Это позволяет управлять оборудованием на самом низком уровне, но требует от разработчика глубокого понимания архитектуры микроконтроллера.

Одним из важных преимуществ программирования микроконтроллеров является детерминированное выполнение операций — время выполнения каждой инструкции известно точно, что критически важно для систем реального времени. Однако это достигается за счёт ограниченных ресурсов: микроконтроллеры имеют мало памяти (обычно килобайты) и работают на сравнительно низких частотах процессора (мегагерцы вместо гигагерц).

Программа для Arduino называется прошивкой (firmware) — она загружается в память микроконтроллера один раз и работает постоянно, в отличие от программ для операционных систем, которые запускаются и останавливаются по необходимости.

В отличие от этого, разработка под операционную систему, как на Lichee RV Dock, происходит на другом уровне абстракции. Здесь программа работает поверх ядра Linux, которое предоставляет абстракцию оборудования через драйверы и системные вызовы. Это упрощает разработку, но добавляет накладные расходы. Операционная система обеспечивает многозадачность — несколько программ могут работать одновременно, разделяя ресурсы процессора. Системы с ОС обычно имеют значительно большие ресурсы: больше памяти, более высокую производительность процессора и развитую файловую систему. Программа в таком контексте представляет собой исполняемый файл, который можно запускать и останавливать по мере необходимости.

10.2.3 Архитектура микроконтроллера ATmega328P

Arduino Uno основана на микроконтроллере ATmega328P: - **Тактовая частота:** 16 МГц - **Флэш-память:** 32 КБ (для программы) - **ОЗУ:** 2 КБ (для данных) - **Энергонезависимая память:** 1 КБ EEPROM - **Периферия:** таймеры, АЦП, UART, SPI, I2C, GPIO

10.2.4 Как заливается прошивка в Arduino?

Процесс программирования Arduino состоит из нескольких этапов. Сначала разработчик пишет код в среде Arduino IDE, используя упрощённый диалект C/C++ с готовыми библиотеками для работы с периферией. Затем среда разработки компилирует код в машинные инструкции для архитектуры AVR, оптимизируя его под ограниченные ресурсы микроконтроллера.

После компиляции происходит загрузка прошивки через USB-порт с использованием встроенного программатора. Как только прошивка успешно загружена, программа начинает выполняться сразу же — микроконтроллер переходит к выполнению кода без необходимости перезагрузки или дополнительных действий.

Загрузчик (bootloader) играет ключевую роль в этом процессе. Это специальная программа, которая постоянно находится в защищённой области памяти микроконтроллера.

При включении Arduino загрузчик первым делом проверяет, не поступила ли команда на загрузку новой прошивки. Если такой команды нет, он просто передаёт управление основной программе. Если же пользователь пытается загрузить новый скетч, загрузчик активирует режим программирования: он принимает данные через последовательный порт (UART), проверяет их целостность, и записывает во флэш-память микроконтроллера. После успешной записи загрузчик выполняет верификацию — сравнивает записанные данные с полученными, и только затем запускает новую программу. Эта система позволяет программировать Arduino без использования внешних программаторов, что значительно упрощает процесс разработки и отладки.

10.2.5 Основные концепции программирования Arduino

Скетч (sketch) — так называется программа для Arduino. Каждый скетч состоит из двух обязательных функций:

```
void setup() {  
    // Выполняется один раз при старте  
    // Инициализация пинов, настройка периферии  
}  
  
void loop() {  
    // Выполняется бесконечно в цикле  
    // Основная логика программы  
}
```

Цифровые входы/выходы (GPIO) — пины общего назначения, которые могут работать в разных режимах, задаваемых функцией `pinMode()`. Базовые режимы:

- **OUTPUT** — пин работает как выход: микроконтроллер управляет напряжением на пине (HIGH — 5 В или LOW — 0 В) для управления светодиодами, реле и другими устройствами
- **INPUT** — пин работает как вход: микроконтроллер считывает внешний сигнал (HIGH или LOW) от кнопок, датчиков и других схем

Помимо базовых, существуют дополнительные режимы, которые настраивают внутренние резисторы микроконтроллера:

- **INPUT_PULLUP** — вход с подтяжкой к питанию. Внутренний резистор (около 20–50 кОм) подтягивает пин к высокому уровню (HIGH). Когда внешнее устройство (например, кнопка) соединяет пин с землёй, читается LOW. Этот режим удобен для кнопок и контактов — не нужен внешний резистор, а по умолчанию пин всегда находится в известном (высоком) состоянии
- **INPUT_PULLDOWN** — вход с подтяжкой к земле (доступен не на всех пинах Arduino Uno, только на некоторых моделях микроконтроллеров). Пин подтягивается к LOW, а внешний сигнал переводит его в HIGH. На ATmega328P этот режим обычно недоступен через `pinMode()` — для подтяжки к земле используется внешний резистор или подтяжка к питанию с инвертированием логики

Прерывания (interrupts) — это важнейший механизм в программировании микроконтроллеров, который позволяет процессору реагировать на внешние или внутренние события практически мгновенно, не тратя время на постоянную проверку состояния в основном цикле программы. Когда происходит событие, на которое настроено прерывание, микроконтроллер приостанавливает выполнение текущего кода, сохраняет его контекст (содержимое регистров и счётчик команд), и немедленно переходит к специальной функции-обработчику. После завершения обработчика микроконтроллер восстанавливает контекст и продолжает выполнение с того же места, где остановился.

Различают два основных типа прерываний. **Аппаратные прерывания** возникают от внешних устройств и периферии микроконтроллера — сигналов на цифровых пинах, таймеров, UART, SPI, I2C и других коммуникационных интерфейсов. **Программные прерывания** инициируются самим кодом через специальные инструкции процессора — они используются для вызова функций операционной системы или приоритетного переключения задач.

Режимы срабатывания аппаратных прерываний: При настройке внешнего прерывания на пине можно выбрать один из четырёх режимов: - **LOW** — прерывание срабатывает постоянно, пока на пине низкий уровень - **CHANGE** — прерывание срабатывает при любом изменении сигнала (с HIGH на LOW и наоборот) - **RISING** — прерывание срабатывает только при переходе с LOW на HIGH (восходящий фронт) - **FALLING** — прерывание срабатывает только при переходе с HIGH на LOW (нисходящий фронт)

Особенности работы с прерываниями в Arduino на ATmega328P:

- Прерывания доступны только на пинах 2 и 3 (для Arduino Uno на ATmega328P) - Обработчик прерывания должен выполняться максимально быстро, чтобы не блокировать основную программу - Внутри обработчика прерывания не рекомендуется использовать функции вроде `delay()`, `Serial.print()` и другие, которые могут зависнуть - Переменные, используемые внутри обработчика и в основном цикле, должны объявляться с ключевым словом `volatile`, чтобы компилятор не оптимизировал их хранение в регистрах процессора вместо памяти

10.3 Практическая часть

10.3.1 Подготовка необходимого оборудования

1. **Arduino Uno** — подключите к компьютеру через USB
2. **Светодиодная мезонинная плата** — вставляется в соответствии с распиновкой Arduino
3. **Соединительные перемычки** — для соединения оборудования
4. **Lichee RV Dock** — с программой генерации меандра на выводе GPIO 144 из лабораторной №9
5. **Преобразователь логических уровней** размещенный на макетной плате

10.3.2 Установка и настройка Arduino IDE

1. Скачайте Arduino IDE из репозитория, путем установки пакета *arduino*
2. Выберите правильную плату: **Инструменты** → **Плата** → **Arduino Uno**
3. Выберите правильный порт: **Инструменты** → **Порт** (например, COM3 или /dev/ttyUSB0)

10.3.3 Первая программа — “Моргающий светодиод”

Создайте новый скетч и напишите простейшую программу:

```
// Подключение светодиода к пину 13
```

```
void setup() {  
  // Настройка пина как выхода  
  pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop() {  
  // Включить светодиод  
  digitalWrite(LED_BUILTIN, HIGH);  
  delay(1000); // Ждать 1 секунду  
  
  // Выключить светодиод
```

```
digitalWrite(LED_BUILTIN, LOW);
delay(1000); // Ждать 1 секунду
}
```

Что происходит: - `pinMode()` — настраивает пин как вход или выход
 - `digitalWrite()` — устанавливает высокий (5В) или низкий (0В) уровень напряжения на пинах микроконтроллера - `delay()` — приостанавливает выполнение на указанное время (в миллисекундах)

10.3.4 Работа с прерываниями

Прерывания позволяют Arduino реагировать на внешние события без постоянной проверки в основном цикле.

Пины 2 и 3 на Arduino Uno могут быть использованы для работы с аппаратными прерываниями.

Программа с прерыванием:

```
// Пин для прерывания (порт 2 поддерживает прерывание INT0)
#define INTERRUPT_PIN 2

// Пин для светодиода
#define LED_PIN 5

// Счётчик прерываний
volatile int interruptCounter = 0;

void setup() {
  // Настройка пина светодиода
  pinMode(LED_PIN, OUTPUT);

  // Настройка пина прерывания как входа
  pinMode(INTERRUPT_PIN, INPUT);

  // Настройка прерывания:
  // - Пин: INTERRUPT_PIN (2)
  // - Функция обработки: handleInterrupt
  // - Режим: FALLING (срабатывает при переходе с HIGH на LOW)
  // Функция digitalPinToInterrupt преобразует номер пина в номер прерывания
```

```

// Таким образом пользовательская функция handleInterrupt привязывается к
// на пине INTERRUPT_PIN в режиме FALLING
attachInterrupt(digitalPinToInterrupt(INTERRUPT_PIN), handleInterrupt, FALLING);

// Инициализация последовательного порта для отладки
Serial.begin(115200);
Serial.println("Arduino готов к работе");
Serial.println("Ожидание прерываний от Lichee...");
}

void loop() {
    // Основной цикл может выполнять другие задачи
    digitalWrite(LED_PIN, HIGH);
    delay(500);
    digitalWrite(LED_PIN, LOW);
    delay(500);

    // Вывод счётчика прерываний
    static int lastCount = 0;
    if (interruptCounter != lastCount) {
        Serial.print("Получено прерываний: ");
        Serial.println(interruptCounter);
        lastCount = interruptCounter;
    }
}

// Функция обработки прерывания
void handleInterrupt() {
    interruptCounter++; // Увеличиваем счётчик
}

```

10.4 Контрольные вопросы

1. В чём ключевое отличие программирования микроконтроллера (bare-metal) от разработки под операционную систему?
2. Какую роль выполняет загрузчик (bootloader) в Arduino?
3. Какие режимы срабатывания аппаратных прерываний доступны на ATmega328P и чем они отличаются?

4. Почему переменные, используемые и в обработчике прерывания, и в основном цикле, должны объявляться с ключевым словом `volatile`?
5. Зачем нужен преобразователь логических уровней при соединении Arduino (5V) с Lichee RV Dock (3.3V)?
6. Почему в обработчике прерывания не рекомендуется использовать функции `delay()` и `Serial.print()`?

10.5 Задание

Ознакомившись с подготовительным материалом, **протестировав и разобравшись в работе элементарных программ** решить следующие подзадачи:

10.5.1 Программа “Светофор”

Создайте программу, имитирующую бегущий огонек на мезонинной плате, путем “мигания” светодиодами, соединенными с пинами 3,5,6,9 и 10. Огонек должен бежать от 3 пина к 10 и обратно, минуя пины, которые не соединены со светодиодами.

10.5.2 Программа “Повторитель меандра”

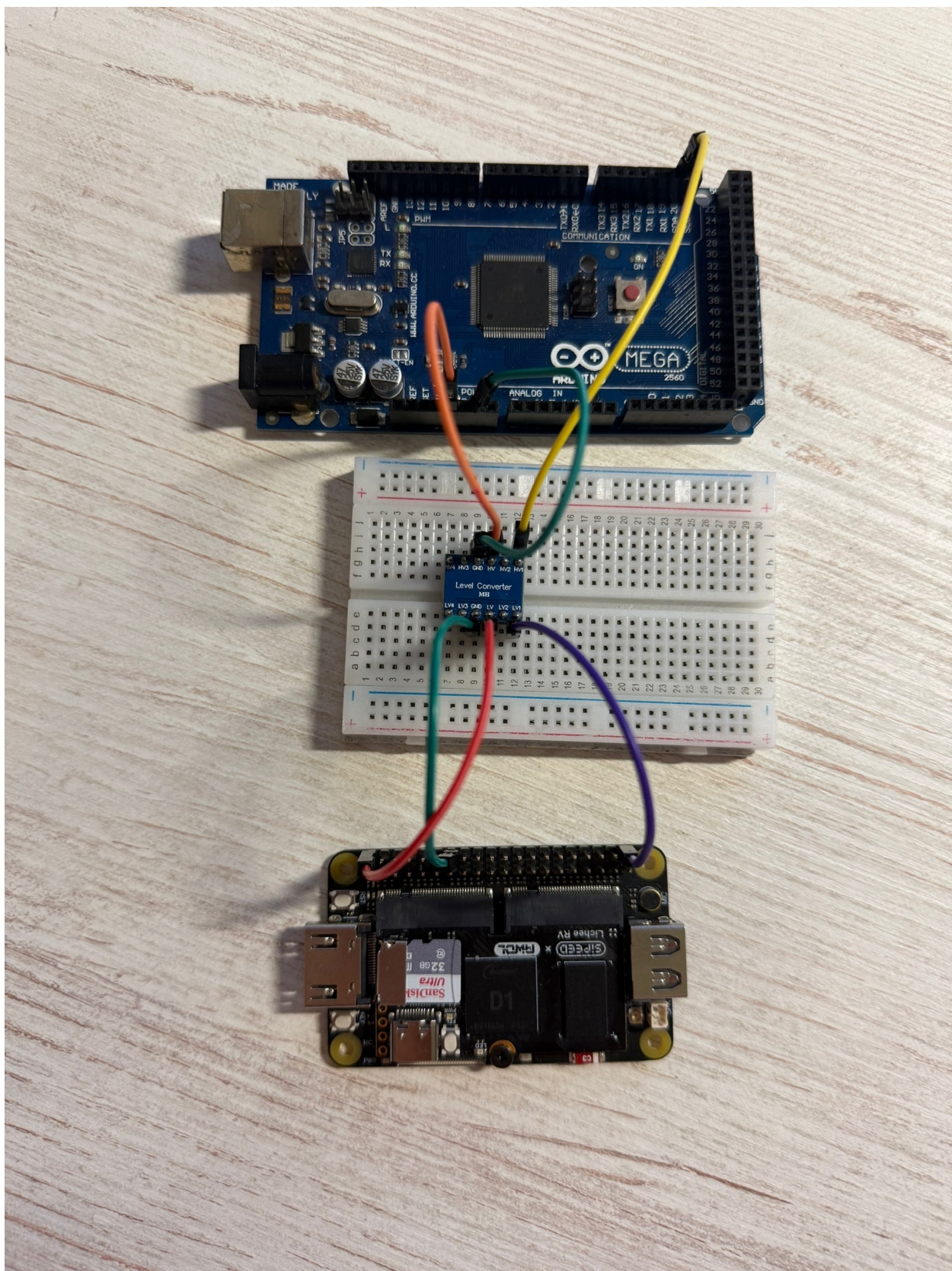
На Lichee RV Dock запустите программу для генерации меандра из лабораторной №9.

Соедините через логический преобразователь уровней пины в следующем соответствии:

- Arduino pin GND → GND пины на преобразователе → Lichee pin GND
- Lichee pin GPIO 144 → LV1/HV1 пин на преобразователе → Arduino pin 2
- Lichee pin 3.3 V → LV/HV пин на преобразователе → Arduino pin 5 V

Преобразователь логических уровней — это схема, которая согласует разные напряжения питания логических сигналов (например, 3.3 В и 5 В), чтобы устройства могли корректно обмениваться данными без повреждения выводов.

Преобразователь в данном случае нужен, чтобы понизить 5 В от Arduino до безопасных 3.3 В для Lichee. На рисунке ниже приведён пример подключения Arduino к Lichee через макетную плату с преобразователем уровней:



Пример подключения устройств

Затем, напишите программу, которая на пине 4 повторяет генерируемый на Lichee меандр, работая с прерыванием на пине №2 в режиме CHANGE.

Анализ сигналов логическим анализатором :

Используя навык работы с логическим анализатором из лабораторной №9, проанализируйте сигналы:

1. **Подключите логический анализатор** к пину GPIO 144 на Lichee и пину 4 на Arduino
2. **Запустите PulseView**
3. **Наблюдайте прерывания** — сигналы от Lichee RV Dock
4. Убедитесь в корректности работы программы для Arduino
5. **Измерьте временные параметры** — длительность импульсов, интервалы, и временную разницу между фронтами сигналов

10.5.3 Демонстрация и отчёт

- Ответить на контрольные вопросы (письменно, в отчёте)
- **Продемонстрировать работу преподавателю**
- Сформировать отчёт о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока**

11 Лабораторная №12. Протокол MQTT

11.1 Цель работы

Изучить протокол MQTT — лёгкий протокол обмена сообщениями для IoT-систем. Освоить архитектуру «издатель-подписчик» (Pub/Sub), научиться настраивать MQTT-брокер Mosquitto на одноплатном компьютере и создавать клиентские приложения на Python и C для публикации и приёма данных.

11.2 Подготовительный материал

11.2.1 Что такое MQTT?

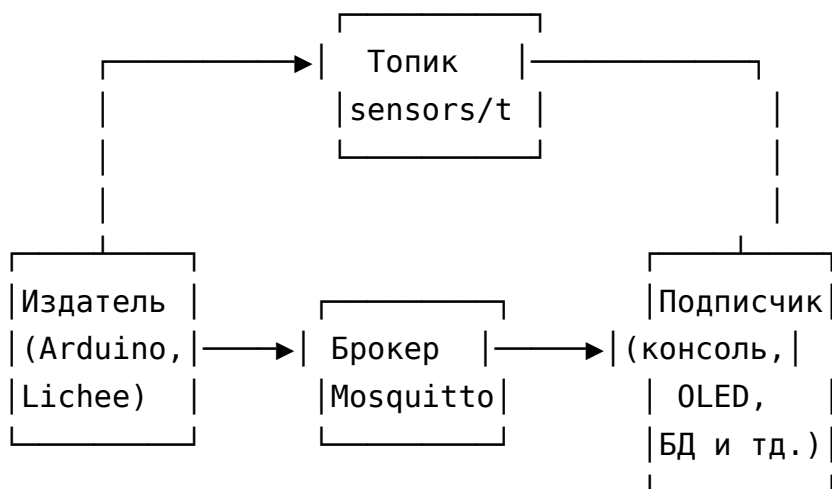
MQTT (Message Queuing Telemetry Transport) — лёгкий протокол обмена сообщениями, разработанный в 1999 году для мониторинга нефтепроводов. Сегодня широко применяется в IoT, умных домах, промышленной автоматизации и телеметрии.

Перечислим основные особенности протокола:

- **Лёгкий протокол** — минимальный заголовок пакета (2 байта), подходит для устройств с ограниченной памятью и медленными каналами
- **Асинхронная модель** — издатель и подписчик не знают друг о друге, взаимодействуют только через брокер
- **Устойчивость к обрывам** — keep-alive, сохранённые сессии, сообщения о разрыве.

11.2.2 Архитектура «издатель-подписчик»

В отличие от клиент-серверной архитектуры, где отправитель напрямую обращается к получателю по адресу, MQTT использует модель Pub/Sub:



Брокер (Broker) — центральный сервер, принимающий сообщения от издателей и доставляющий их подписчикам. В курсе брокером выступает Lichee RV Dock с установленным Mosquitto.

Издатель (Publisher) — клиент, отправляющий данные в топик брокера. Издатель не знает, кто и когда получит его сообщение.

Подписчик (Subscriber) — клиент, получающий сообщения из одного или нескольких топиков. Может подписаться на уже работающий поток данных без каких-либо изменений в издателе.

11.2.3 Топики

Топик (topic) — строковый адрес в иерархической структуре, разделённой символом /. Примеры:

Топик	Данные
home/livingroom/temperature	Температура в гостиной
home/kitchen/humidity	Влажность на кухне
lichee/stats/cpu	Загрузка ЦП одноплатника
sensors/bme280	Все данные с датчика BME280

Wildcard-подписки позволяют подписываться на группы топиков:

- **+** — заменяет ровно один уровень иерархии. Подписка `home/+/temperature` получит данные о температуре из всех комнат
- **#** — заменяет любое количество уровней (только в конце). Подписка `home/#` получит все сообщения из всех комнат и подтопиков дома

11.2.4 Качество обслуживания (QoS)

MQTT предоставляет три уровня гарантии доставки сообщений:

Уровень	Название	Описание	Пример использования
QoS 0	At most once	Сообщение отправляется один раз без подтверждения. Возможна потеря.	Телеметрия: температура раз в минуту
QoS 1	At least once	Брокер подтверждает получение (PUBACK). Возможны дубликаты.	Команды управления освещением
QoS 2	Exactly once	Четырёхэтапная транзакция, списание рукопожатие. Гарантирует ровно одну доставку.	Транзакции, списание средств

В рамках данной лабораторной работы используется QoS 0 — самый простой и быстрый режим.

11.2.5 Retained-сообщения

При публикации с флагом `retain = true` брокер сохраняет последнее сообщение в топике и автоматически отправляет его каждому ново-

му подписчику. Это удобно для статусов, которые меняются редко: новый клиент немедленно получает актуальное значение, не дожидаясь следующей публикации.

11.2.6 Will-сообщения

Will Message задаётся при подключении клиента к брокеру. Если клиент отключается нештатно (обрыв связи, таймаут keep-alive), брокер автоматически публикует заданное will-сообщение. Это позволяет системе мониторинга узнать, что устройство «отвалилось».

11.2.7 MQTT-брокер Mosquitto

Mosquitto — популярный открытый MQTT-брокер, реализующий протоколы MQTT. Доступен в репозиториях ALT Linux.

Установка и запуск:

```
# apt-get install mosquitto
# systemctl start mosquitto
# systemctl enable mosquitto
```

Проверка работоспособности командной строкой:

```
# В первом терминале – подписчик (ждёт сообщения):
$ mosquitto_sub -h localhost -t "test/topic"

# Во втором терминале – издатель (отправляет сообщение):
$ mosquitto_pub -h localhost -t "test/topic" -m "Hello MQTT!"
```

В первом терминале должно появиться Hello MQTT!.

11.2.8 Python-клиент paho-mqtt

Eclipse Paho — библиотека для работы с MQTT на разных языках. Для Python:

```
# apt-get install python3-module-paho
```

Основные callback-функции:

```

import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc):
    """Вызывается при подключении к брокеру. rc == 0 – успех."""
    print(f"Connected: {rc}")

def on_message(client, userdata, msg):
    """Вызывается при получении сообщения из подписанного топика."""
    print(f"{msg.topic}: {msg.payload.decode()}")

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect("localhost", 1883)      # адрес брокера, порт
client.subscribe("topic/name")        # подписка на топик
client.loop_forever()                 # бесконечный цикл обработки

```

Для публикации используется метод `publish()`:

```
client.publish("topic/name", "data_string")
```

11.2.9 С-клиент libmosquitto

Для одноплатника доступна библиотека `libmosquitto` — нативная С-реализация клиента MQTT:

```
# apt-get install libmosquitto libmosquitto-devel
```

11.3 Практическая часть

11.3.1 Шаг 1: Установка и настройка брокера Mosquitto на Lichee

Брокер будет запущен на Lichee RV Dock. На ПК установите только клиентские утилиты для тестирования.

1. **На Lichee:** установите Mosquitto и клиентские утилиты:

```
# apt-get install mosquitto
```

2. **На ПК (ALT Linux):** установите Mosquitto — пакет включает клиентские утилиты mosquitto_pub/mosquitto_sub для тестирования:

```
# apt-get install mosquitto
```

3. **На Lichee:** запустите брокер и добавьте в автозагрузку:

```
# systemctl start mosquitto
```

```
# systemctl enable mosquitto
```

4. **На Lichee:** настройте брокер для приёма внешних подключений. По умолчанию Mosquitto слушает только localhost. Добавьте в файл /etc/mosquitto/mosquitto.conf строки:

```
listener 1883 0.0.0.0
allow_anonymous true
```

Перезапустите брокер:

```
# systemctl restart mosquitto
```

Проверьте, что порт 1883 слушается на всех интерфейсах:

```
$ ss -tlnp | grep 1883
```

Вывод должен содержать 0.0.0.0:1883 или *:1883.

5. **Узнайте IP-адрес Lichee** (понадобится для подключения подписчика с ПК):

```
$ ip a | grep "inet " | grep -v 127.0.0.1
```

или:

```
$ hostname -i
```

Запишите этот IP-адрес — далее будем обозначать его <IP_LICHEE>.

6. **Проверка связности с ПК:**

С ПК проверьте доступность Lichee:

```
$ ping <IP_LICHEE>
```

Проверьте, что порт 1883 на Lichee открыт:

```
$ telnet <IP_LICHEE> 1883
```

Если соединение устанавливается (в telnet появится Connected или escape-символ) — брокер доступен.

7. Протестируйте Pub/Sub между Lichee и ПК:

Терминал 1 — на ПК, подписчик:

```
$ mosquitto_sub -h <IP_LICHEE> -t "test/hello"
```

Терминал 2 — на Lichee (по SSH), издатель:

```
$ mosquitto_pub -h localhost -t "test/hello" -m "Hello from Lichee!"
```

В терминале на ПК должно появиться: Hello from Lichee!. Брокер работает, сеть настроена.

11.3.2 Шаг 2: Python-издатель системных метрик

Создайте скрипт mqtt_publisher.py, который читает загрузку CPU и использование RAM и публикует их в топик lichee/stats:

```
#!/usr/bin/env python3
```

```
import paho.mqtt.client as mqtt
```

```
import json
```

```
import time
```

```
TOPIC = "lichee/stats"
```

```
INTERVAL = 2.0
```

```
def get_cpu_usage():
```

```
    """Возвращает загрузку CPU в процентах (от 0 до 100)."""
```

```
    with open("/proc/stat") as f:
```

```
        fields = f.readline().split()
```

```
    user, nice, system, idle = map(int, fields[1:5])
```

```
    total = user + nice + system + idle
```

```

# Расчёт по дельте между вызовами
if not hasattr(get_cpu_usage, "prev"):
    get_cpu_usage.prev = (total, idle)
    return 0.0

prev_total, prev_idle = get_cpu_usage.prev
get_cpu_usage.prev = (total, idle)

diff_total = total - prev_total
diff_idle = idle - prev_idle
return 100.0 * (1.0 - diff_idle / diff_total) if diff_total > 0 else 0.0

def get_ram_usage():
    """Возвращает использование RAM в процентах (от 0 до 100)."""
    total = free = 0
    with open("/proc/meminfo") as f:
        for line in f:
            if line.startswith("MemTotal:"):
                total = int(line.split()[1])
            elif line.startswith("MemAvailable:"):
                free = int(line.split()[1])
    return 100.0 * (total - free) / total if total > 0 else 0.0

def main():
    client = mqtt.Client()
    client.connect("localhost", 1883)
    client.loop_start()

    print(f"Publishing to '{TOPIC}' every {INTERVAL}s. Ctrl+C to stop.")
    try:
        while True:
            cpu = get_cpu_usage()
            ram = get_ram_usage()
            payload = json.dumps({"cpu": round(cpu, 1), "ram": round(ram, 1)})
            client.publish(TOPIC, payload)
            print(f"Published: {payload}")
            time.sleep(INTERVAL)
    except KeyboardInterrupt:
        print("Stopped.")

```

```

    finally:
        client.loop_stop()
        client.disconnect()

if __name__ == "__main__":
    main()

```

Объяснение: - `get_cpu_usage()` читает `/proc/stat`, вычисляя загрузку как 100% - доля простоя между двумя вызовами. Первый вызов возвращает 0 (базовый замер), последующие — реальное значение - `get_ram_usage()` читает `/proc/meminfo`, находит поля `MemTotal` и `MemAvailable`, вычисляет процент занятой памяти - Данные упаковываются в JSON и публикуются каждые 2 секунды - `loop_start()` запускает сетевой цикл `raho` в фоновом потоке, не блокируя основной

Запустите скрипт:

```
$ python3 mqtt_publisher.py
```

Оставьте работать — на следующем шаге мы подпишемся на этот топик.

11.3.3 Шаг 3: Python-подписчик на ПК

Скрипт подписчика будет запущен на вашем ПК. Он подключается к брокеру на Lichee по IP-адресу и получает данные из топика `lichee/stats`.

Создайте на ПК скрипт `mqtt_subscriber.py`:

```

#!/usr/bin/env python3

import paho.mqtt.client as mqtt
import json

# IP-адрес Lichee RV Dock (узнайте командой "hostname -i" на Lichee)
BROKER = "192.168.1.50"
TOPIC = "lichee/stats"

def on_connect(client, userdata, flags, rc):
    print(f"Connected to broker (rc={rc})")
    client.subscribe(TOPIC)

```

```

    print(f"Subscribed to '{TOPIC}'")

def on_message(client, userdata, msg):
    try:
        data = json.loads(msg.payload.decode())
        cpu = data.get("cpu", "?")
        ram = data.get("ram", "?")
        print(f"CPU: {cpu:5.1f}%    RAM: {ram:5.1f}%")
    except json.JSONDecodeError:
        print(f"Raw: {msg.payload.decode()}")

def main():
    client = mqtt.Client()
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, 1883)
    print(f"Connected to MQTT broker at {BROKER}")
    print("Listening for messages...")
    client.loop_forever()

if __name__ == "__main__":
    main()

```

Объяснение: - BROKER — IP-адрес Lichee RV Dock в вашей локальной сети. Замените "192.168.1.50" на реальный адрес, полученный командой `hostname -i` на Lichee - `on_connect` вызывается при успешном подключении — здесь выполняется подписка на топик - `on_message` вызывается при получении сообщения: парсинг JSON, извлечение `cpu` и `ram`, форматированный вывод - `loop_forever()` блокирует поток и обрабатывает входящие MQTT-пакеты бесконечно

Установка `paho-mqtt` на ПК (если не установлена):

```
# apt-get install python3-module-paho
```

Запуск:

1. Убедитесь, что издатель `mqtt_publisher.py` запущен на Lichee (шаг 2)
2. Запустите подписчик на ПК:

```
$ python3 mqtt_subscriber.py
```


В консоли должно появиться:

```
Connected to broker (rc=0)
Subscribed to 'lichee/stats'
CPU:  12.3%   RAM:  45.7%
CPU:  15.1%   RAM:  45.8%
...
```

11.3.4 Шаг 4: С-клиент-издатель (нативная компиляция на Lichee)

MQTT-клиент можно написать не только на Python, но и на С с использованием библиотеки `libmosquitto`. Это даёт меньший размер бинарного файла и лучшую производительность.

4а. Установка библиотек на одноплатнике (для нативной компиляции):

```
# apt-get install libmosquitto libmosquitto-devel
```

4б. Исходный код (`mqtt_c_publisher.c`):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <mosquitto.h>

#define MQTT_HOST    "localhost"
#define MQTT_PORT    1883
#define MQTT_TOPIC   "lichee/stats"
#define KEEPALIVE    60

static float get_cpu_usage(void) {
    FILE *fp = fopen("/proc/stat", "r");
    if (!fp) return -1;

    unsigned long user, nice, system, idle;
    fscanf(fp, "cpu %lu %lu %lu %lu", &user, &nice, &system, &idle);
    fclose(fp);
}
```

```

    unsigned long total = user + nice + system + idle;
    static unsigned long prev_total = 0, prev_idle = 0;
    float usage = 0.0;

    if (prev_total > 0) {
        float diff_idle = idle - prev_idle;
        float diff_total = total - prev_total;
        usage = 100.0 * (1.0 - diff_idle / diff_total);
    }
    prev_total = total;
    prev_idle = idle;
    return usage;
}

static float get_ram_usage(void) {
    FILE *fp = fopen("/proc/meminfo", "r");
    if (!fp) return -1;

    unsigned long total = 0, available = 0;
    char line[128];
    while (fgets(line, sizeof(line), fp)) {
        if (strstr(line, "MemTotal:"))
            sscanf(line, "MemTotal: %lu kB", &total);
        if (strstr(line, "MemAvailable:"))
            sscanf(line, "MemAvailable: %lu kB", &available);
    }
    fclose(fp);
    return (total > 0) ? 100.0 * (total - available) / total : -1;
}

int main(void) {
    mosquitto_lib_init();

    struct mosquitto *mosq = mosquitto_new(NULL, true, NULL);
    if (!mosq) {
        fprintf(stderr, "Error: mosquitto_new failed\n");
        return 1;
    }

    if (mosquitto_connect(mosq, MQTT_HOST, MQTT_PORT, KEEPALIVE)) {

```

```

        fprintf(stderr, "Error: cannot connect to broker\n");
        mosquitto_destroy(mosq);
        mosquitto_lib_cleanup();
        return 1;
    }

    printf("C MQTT publisher started.\n");
    printf("Publishing to '%s'. Press Ctrl+C to stop.\n", MQTT_TOPIC);

    char payload[128];
    while (1) {
        float cpu = get_cpu_usage();
        float ram = get_ram_usage();

        if (cpu < 0 || ram < 0) {
            fprintf(stderr, "Error reading system stats\n");
            sleep(1);
            continue;
        }

        snprintf(payload, sizeof(payload),
                 "{\\"cpu\\":%.1f,\\"ram\\":%.1f}", cpu, ram);

        int ret = mosquitto_publish(mosq, NULL, MQTT_TOPIC,
                                    strlen(payload), payload, 0, false);
        if (ret != MOSQ_ERR_SUCCESS)
            fprintf(stderr, "Publish error: %s\n", mosquitto_strerror(ret));
        else
            printf("Sent: %s\n", payload);

        sleep(2);
    }

    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();
    return 0;
}

```

4с. Нативная компиляция на Lichee:

```
$ gcc -o mqtt_c_publisher mqtt_c_publisher.c -lmosquitto
```

```
$ ./mqtt_c_publisher
```

Исходный файл `mqtt_c_publisher.c` копируется на Lichee через `scp`, компилируется нативно командой выше и запускается прямо на одноплатнике.

11.4 Контрольные вопросы

1. В чём преимущества архитектуры «издатель-подписчик» по сравнению с клиент-серверной для IoT-систем?
2. Какие уровни QoS существуют в MQTT и в каких сценариях каждый из них целесообразно использовать?
3. Как обеспечивается безопасность в MQTT? Какие механизмы аутентификации и шифрования поддерживаются?
4. Чем отличается MQTT от других протоколов IoT, таких как CoAP или HTTP/2?
5. Как организовать иерархию топиков для системы умного дома?
6. Какие проблемы могут возникнуть при использовании MQTT в сетях с нестабильным соединением и как их решить?
7. Как масштабировать MQTT-инфраструктуру при увеличении количества устройств?
8. В чём преимущества использования MQTT поверх WebSocket для веб-приложений?
9. Как реализовать retained messages и will messages в MQTT и для чего они используются?
10. Какие библиотеки для работы с MQTT существуют для различных платформ?

11.5 Задание

Ознакомившись с подготовительным материалом, выполните следующие подзадачи:

1. **Базовая задача.** Установите и запустите MQTT-брокер Mosquitto на Lichee RV Dock. Настройте брокер на приём подключений со всех сетевых интерфейсов (listener 1883 0.0.0.0). Протестируйте публикацию и подписку между ПК и Lichee через утилиты `mosquitto_pub` и `mosquitto_sub`.

2. **Основная задача.** Реализуйте распределённую систему мониторинга загрузки одноплатника:
- На Lichee: напишите Python-скрипт-издатель, публикующий загрузку CPU и использование RAM (в процентах) в топик `lichee/stats` раз в 2 секунды. Данные передавайте в формате JSON: `{"cpu": 12.3, "ram": 45.7}`.
 - На ПК: напишите Python-скрипт-подписчик, подключающийся к брокеру на Lichee (по IP-адресу), принимающий данные из топика `lichee/stats` и выводящий их в консоль в читаемом виде: CPU: 12.3% RAM: 45.7%.
 - Продемонстрируйте одновременную работу издателя (на Lichee) и подписчика (на ПК).
3. **Дополнительная часть (С-клиент).** Напишите аналог MQTT-издателя на языке C, публикующий те же системные метрики. Выполните нативную компиляцию программы и проверьте её работу на Lichee.
4. **Ответить на контрольные вопросы (письменно, в отчёте).**
5. **Продемонстрировать работу преподавателю.**
6. Сформировать отчёт о выполнении поставленных задач .doc и **выслать на почту преподавателя до обозначенного срока.**

11.6 Полезные ссылки

- Пример Python-издателя — публикация CPU/RAM в `lichee/stats` раз в 2 секунды
- Пример Python-подписчика — подключение к брокеру, приём данных из `lichee/stats`
- Пример C-издателя (`libmosquitto`) — аналог на языке C с использованием `libmosquitto`
- Официальный сайт MQTT
- Документация Mosquitto
- Eclipse Paho Python
- Документация `libmosquitto`

12 Лабораторная №13. Итоговое проектное задание по вариантам

12.1 Цель работы

Самостоятельно спроектировать и реализовать распределённую систему, объединяющую одноплатный компьютер Lichee RV Dock, микроконтроллер Arduino Uno и графическое приложение на ПК. Транспорт между компонентами — протокол MQTT (брокер Mosquitto на Lichee).

12.2 Порядок выполнения

1. Каждый студент получает **один из десяти вариантов** задания
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino .ino, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для C-частей)
5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений

Охват лабораторных: №7 (SPI), №8 (I2C OLED), №10 (Arduino), №11 (Arduino + BME280 + SPI), №12 (MQTT).

12.3 Вариант 1. Метеостанция — сбор и визуализация данных с датчика BME280

12.3.1 Описание

Собрать распределённую систему, в которой данные с датчика BME280 (температура, влажность, давление) передаются от Arduino к одноплатнику Lichee, далее через MQTT на ПК и отображаются в реальном времени в графическом интерфейсе.

12.3.2 Цепочка

BME280 — I2C → Arduino Uno — SPI(slave) → Lichee RV Dock — MQTT → ПК (GUI)

12.3.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка Arduino читает BME280 по I2C (библиотека `GyverBME280`), передаёт три значения float (температура °C, влажность %, давление гПа) по SPI в режиме slave на Lichee. Протокол — серия однобайтовых транзакций.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Python-скрипт на Lichee принимает данные от Arduino через `/dev/spidev1.0`, распаковывает `struct.unpack("<fff", ...)` и публикует их в MQTT-топик `sensors/bme280` в формате JSON:

```
{"temperature": 25.1, "humidity": 48.3, "pressure": 1004.5, "timestamp": 1611111111}
```

3. **I2C OLED (лаб. №8).** Дополнительно — дублировать данные на OLED-экран SSD-1306 через I2C на Lichee (три строки: температура, влажность, давление).
4. **ПК — GUI-приложение.** Python-приложение, подключающееся к MQTT-брокеру на Lichee и отображающее:
 - Текущие значения трёх параметров — крупным шрифтом в отдельных блоках
 - Три графика (`matplotlib`, встроенный в GUI) с историей за последние 60 секунд (ось X — время, ось Y — значение)

- Статус подключения: индикатор (зелёный/красный кружок) — получает ли система данные. Статус менять по тайм-ауту: если данные не поступали более 5 секунд — «нет данных»

12.3.4 Ключевые точки проверки

- Физически подключённый BME280 выдаёт реалистичные значения
- При отключении датчика от Arduino GUI показывает «нет данных» (красный индикатор)
- При отключении Arduino от Lichee — аналогично
- Графики обновляются плавно, история не теряется при свёртывании окна

12.4 Вариант 2. Пульт управления светодиодами — remote control через MQTT и SPI

12.4.1 Описание

Через графический интерфейс на ПК управлять светодиодами на мезонинной плате Arduino, передавая команды по цепочке MQTT → Lichee → SPI → Arduino.

12.4.2 Цепочка

ПК (GUI) —MQTT→ Lichee RV Dock —SPI(master)→ Arduino Uno —GPIO→ LED-мезонин

12.4.3 Требования

1. **Arduino — приём команд по SPI (лаб. №10, №11).** Прошивка работает в режиме SPI-slave и управляет светодиодами на пинах 3, 5, 6, 9, 10 (мезонинная плата). Команды приходят от Lichee-мастера. Протокол команд студент проектирует самостоятельно (например: байт команды + байт данных; или 5 бит, соответствующих пяти светодиодам).

Поддерживаемые операции для каждого светодиода:

- включить / выключить (индивидуально)
- «бегущий огонёк» вперёд (пины 3→10→3)
- «бегущий огонёк» назад (пины 10→3→10)
- остановить анимацию

Скорость бегущего огонька задаётся параметром (задержка в миллисекундах).

2. **Lichee — MQTT-подписчик + SPI-мастер (лаб. №12).** Python-скрипт подписывается на MQTT-топик led/command, получает JSON-команды и транслирует их в SPI-транзакции к Arduino.

Формат JSON-команды (примеры):

```
{"pin": 3, "action": "on"}
{"pin": 5, "action": "off"}
{"action": "animation", "direction": "forward", "delay_ms": 150}
{"action": "stop"}
```

3. **ПК — GUI-приложение.** Окно с элементами управления:

- 5 тумблеров (checkbox/toggle) для индивидуальных светодиодов с подписями «LED 3», «LED 5», ...
- Кнопка «Бегущий вперёд» и кнопка «Бегущий назад»
- Слайдер задержки анимации (50–500 мс)
- Кнопка «Стоп»
- При изменении любого элемента GUI публикуется соответствующая MQTT-команда

12.4.4 Ключевые точки проверки

- Каждый тумблер индивидуально включает/выключает свой светодиод
 - Бегущий огонёк работает в обоих направлениях
 - Смена задержки на слайдере меняет скорость анимации
 - «Стоп» останавливает анимацию и оставляет текущее состояние светодиодов
-

12.5 Вариант 3. Системный монитор одноплатника — dashboard

12.5.1 Описание

Создать дашборд (панель мониторинга), отображающий в реальном времени системные метрики одноплатника Lichee: загрузку CPU, использование RAM, занятость корневого раздела, сетевой трафик. Метрики собираются на Lichee и публикуются по MQTT. GUI на ПК отображает их в виде приборных панелей и графиков.

12.5.2 Цепочка

Lichee RV Dock (/proc, /sys) —MQTT→ ПК (GUI)

12.5.3 Требования

1. **Lichee — сборщик метрик (лаб. №12).** Python-скрипт-издатель, публикующий в MQTT-топик `lichee/stats` JSON-пакет раз в 2 секунды:

```
{  
  "cpu_percent": 12.3,  
  "ram_percent": 45.7,  
  "disk_percent": 32.1,  
  "rx_bytes_per_sec": 1024,
```

```

"tx_bytes_per_sec": 512,
"uptime_seconds": 12345,
"hostname": "lichee-rv",
"kernel": "6.16.0"
}

```

Источники данных:

- CPU: /proc/stat
 - RAM: /proc/meminfo
 - Диск: os.statvfs('/')
 - Сеть: /sys/class/net/<интерфейс>/statistics/rx_bytes, tx_bytes (дельта между опросами)
 - Uptime: /proc/uptime
 - Hostname: socket.gethostname()
 - Версия ядра: os.uname().release
2. **Дополнительно.** Написать сборщик метрик на языке C с использованием только системных вызовов и файлов /proc, /sys. Скомпилировать под RISC-V (кросс-компиляция с x86_64). По быстродействию сравнить с Python-версией.
3. **ПК — GUI-приложение.** Интерфейс дашборда содержит:
- 4 «приборных панели» (gauge / стрелочный индикатор) для CPU, RAM, диска, сети. Каждая панель показывает текущее значение в процентах и имеет цветовое кодирование (зелёный < 50%, жёлтый 50–80%, красный > 80%)
 - Строка статуса в нижней части окна: uptime, hostname, версия ядра
 - График истории (опционально): загрузка CPU и RAM за последние 60 секунд
 - Кнопка «Обновить сейчас» — публикует команду в lichee/cmd ({"action": "refresh"})

12.5.4 Ключевые точки проверки

- Все четыре панели обновляются и показывают адекватные значения
- При нагрузке на Lichee (запуск stress или dd) значения меняются
- Кнопка «Обновить сейчас» вызывает немедленную публикацию
- Строка статуса показывает реальные uptime и hostname

12.6 Вариант 4. Генератор сигналов с управлением по MQTT — практическая верификация анализатором

12.6.1 Описание

С графического интерфейса на ПК задать параметры цифрового сигнала (меандра). Lichee генерирует сигнал на GPIO-пине. Студент подключает логический анализатор к этому пину, захватывает сигнал в PulseView и подтверждает соответствие заданным параметрам.

12.6.2 Цепочка

ПК (GUI) —MQTT→ Lichee (libgpiod2) —GPIO→ Логический анализатор (fx2lafv)

12.6.3 Требования

1. **Lichee — генератор сигнала (лаб. №9).** Python-скрипт подписывается на MQTT-топик `gpio/control`, принимает JSON-команды и управляет GPIO-пином (PE16, линия 144) через `libgpiod2`.

Формат JSON-команды:

```
{"action": "start", "gpio_line": 144, "freq_hz": 1000, "duty_percent": 50}
{"action": "stop"}
```

После получения "start" скрипт в отдельном потоке генерирует меандр. При "stop" поток останавливается, пин переводится в низкий уровень. При повторном "start" без "stop" параметры обновляются без разрыва сигнала.

2. **ПК — GUI-приложение.** Окно с элементами управления:

- Выпадающий список с доступными GPIO-линиями Lichee (линии 144, 145, 146 — PE16, PE17, PE18)

- Поле ввода частоты (Гц) — от 1 до 100 000 Гц, с проверкой корректности ввода. Значение по умолчанию: 1000 Гц
- Слайдер или поле ввода скважности (%) — от 1 до 99%. По умолчанию: 50%
- Кнопка «Старт» и кнопка «Стоп»
- Предпросмотр сигнала — на Canvas отрисовывается визуализация меандра (два периода) с подписями периода и длительности уровней

3. Верификация (лаб. №9). Подключить логический анализатор (fx2lafw) к заданному GPIO-пину Lichee. Запустить захват в PulseView (частота дискретизации $\geq 4\times$ от частоты сигнала, но не менее 12 МГц). Измерить период и скважность сигнала и сравнить с заданными в GUI значениями.

12.6.4 Ключевые точки проверки

- Сигнал на GPIO появляется при нажатии «Старт» и пропадает при «Стоп»
- Частота и скважность, измеренные в PulseView, соответствуют заданным (погрешность в пределах 5% для частот до 10 кГц)
- Изменение параметров через GUI во время работы генератора обновляет сигнал без остановки
- Предпросмотр на Canvas корректно отображает два периода меандра с верными метками времени

12.7 Вариант 5. Светодиодный проигрыватель мелодий

12.7.1 Описание

Реализовать визуальный проигрыватель мелодий, в котором на ПК в графическом редакторе задаётся «партитура» — последовательность включений светодиодов во времени. Мелодия передаётся по цепочке MQTT → Lichee → SPI → Arduino и проигрывается на ленте из пяти светодиодов мезонинной платы (пины 3, 5, 6, 9, 10).

12.7.2 Цепочка

ПК (GUI-редактор) —MQTT—► Lichee RV Dock —SPI(master)—► Arduino Uno —GPIO

12.7.3 Ключевое проектное решение

Arduino получает только один байт — битовую маску пяти светодиодов (биты 0–4). Вся логика темпа (BPM) и таймингов размещается на Lichee, что минимизирует Arduino-код и упрощает SPI-протокол до однобайтовых транзакций.

12.7.4 Требования

1. **Arduino — прошивка SPI-slave (лаб. №10, №11).** Минимальный скетч в режиме SPI-slave. Принимает один байт от Lichee-мастера и устанавливает пины в соответствии с битами принятого байта. SPI: режим slave, пины MISO(12)/MOSI(11)/SCK(13)/SS(10). При получении 0x00 — гасит всё.
2. **Lichee — MQTT-подписчик + SPI-транслятор (лаб. №12).** Python-скрипт, подписывающийся на MQTT-топик melody/command. При "play" — сохраняет массив шагов и BPM, вычисляет длительность одного шага и в отдельном потоке отправляет битовые маски по SPI с нужной задержкой. После последнего шага отправляет 0x00 и публикует статус "stopped". SPI: /dev/spidev1.0, mode 0, скорость 100 кГц.
3. **ПК — GUI-редактор мелодии.** Окно приложения содержит:
 - Редактор (Canvas с сеткой): 5 строк (LED1–5), колонки — шаги (до 64). Клик по ячейке — переключение вкл/выкл
 - Слайдер темпа (BPM): 60–240, по умолчанию 120
 - Кнопки Play, Stop
 - Выпадающий список «Загрузить пример»: 3 встроенных мелодии
 - Кнопка «Сохранить» / «Загрузить» — JSON-файл
 - Индикация проигрывания: вертикальная красная линия движется по текущему шагу

MQTT-команды в топик melody/command:

```

{"action": "play", "bpm": 120, "melody": [[1,0,0,1,0], [0,0,1,0,0], ..
{"action": "stop"}

```

12.7.5 Ключевые точки проверки

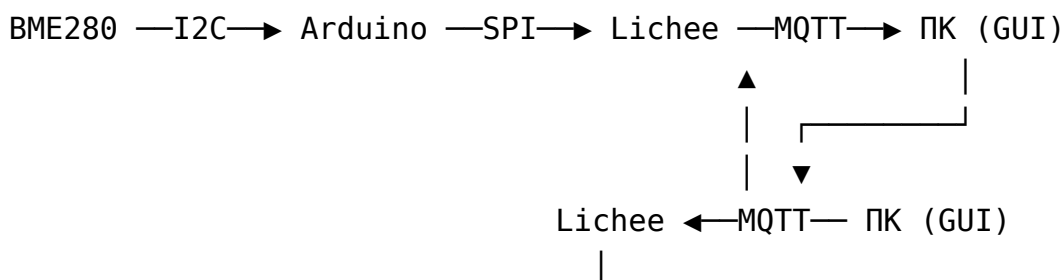
- При Play светодиоды загораются в точном соответствии с рисунком в редакторе
- Красная линия в GUI движется синхронно с физическими светодиодами
- При Stop всё гаснет
- Изменение BPM на слайдере ускоряет/замедляет проигрывание
- Сохранение и загрузка JSON-файла работают корректно

12.8 Вариант 6. Система мониторинга с оповещением на I2C OLED

12.8.1 Описание

Расширение варианта №1 (Метеостанция). Данные с датчика BME280 идут по цепочке Arduino → SPI → Lichee → MQTT → GUI на ПК. Дополнительно оператор задаёт в GUI пороговые значения для каждого параметра. При превышении порога GUI сигнализирует тревогу. Оператор жмёт кнопку «Оповестить на OLED» — по обратному каналу MQTT → Lichee на I2C-экран SSD-1306 выводится тревожное сообщение с текущим значением.

12.8.2 Цепочка



12.8.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1: читает BME280 по I2C (GyverBME280), передаёт три float по SPI-slave на Lichee.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Принимает данные по SPI, публикует JSON в топик `sensors/bme280`.
3. **Lichee — подписчик на тревоги (лаб. №8, №12).** Отдельный Python-скрипт, подписывающийся на топик `alert/oled`. При получении сообщения выводит текст на SSD-1306 через I2C (luma-oled). Через 5 секунд автоматически очищает экран.
4. **ПК — GUI-приложение.** Все элементы варианта №1 плюс:
 - Поля ввода порогов: `T_max`, `H_max`, `P_min`, `P_max` со значениями по умолчанию
 - Индикация тревоги: при превышении порога строка параметра подсвечивается красным и мигает
 - Кнопка «Оповестить на OLED»: активна только когда есть активная тревога
 - Сброс тревоги: когда значение возвращается в пределы порога + гистерезис

12.8.4 Ключевые точки проверки

- При превышении порога GUI подсвечивает соответствующую строку красным
 - Кнопка «Оповестить на OLED» активна только при тревоге; при нажатии сообщение появляется на физическом экране SSD-1306
 - Через 5 секунд экран возвращается к обычному отображению
 - При возврате значения в норму тревога сбрасывается с гистерезисом
-

12.9 Вариант 7. Генератор загрузочных образов — консольный сборщик SD-карты

12.9.1 Описание

Создать консольное приложение на Python, которое из готовых компонентов автоматически собирает загрузочную SD-карту для Lichee RV Dock с заданной конфигурацией. Готовые ядра, Device Tree Blob, rootfs и загрузчик предоставляются преподавателем.

12.9.2 Цепочка

ПК (Python-скрипт) —fdisk/dd/mkfs/cp→ SD-карта

12.9.3 Требования

Скрипт `build_image.py` с интерфейсом командной строки:

```
$ python3 build_image.py --device /dev/sdX --config spi [--dry-run]
```

Конфигурация	Ядро	Device Tree	Поддерживаемые интерфейсы
basic	Image	sun20i-d1-lichee-rv-dock.dtb	WiFi
spi	Image-spi	sun20i-d1-lichee-rv-dock-spi.dtb	WiFi + SPI
i2c	Image-i2c	sun20i-d1-lichee-rv-dock-i2c.dtb	WiFi + I2C
full	Image-spi	.dtb с SPI + I2C	WiFi + SPI + I2C

Действия программы (по порядку):

1. Проверка: `--device` существует, является блочным устройством и не примонтирован

2. Разбивка: `fdisk` создаёт таблицу разделов DOS (первый раздел 100 МБ FAT32, второй — остаток ext4)
3. Запись загрузчика: `dd if=u-boot-sunxi-with-spl.bin of=<device> bs=1k seek=8`
4. Форматирование: `mkfs.vfat -F 32 <device>1, mkfs.ext4 <device>2`
5. Копирование файлов на BOOT и распаковка rootfs
6. `sync` и уведомление о завершении

Вывод программы — человекочитаемый лог каждого шага:

```
[INFO] Checking device /dev/sdb ... OK (not mounted)
[INFO] Selected config: spi
[INFO] Step 1/6: Partitioning ...
[INFO] Step 2/6: Writing U-Boot (seek=8) ...
...
[INFO] Done. SD card is ready at /dev/sdb
```

12.9.4 Ключевые точки проверки

- Карта, собранная с конфигурацией `basic`, загружается, работает WiFi
- С конфигурацией `spi` — загружается, WiFi и SPI работают (проверить `ls /dev/spidev*`)
- С конфигурацией `i2c` — загружается, WiFi и I2C работают (проверить `ls /dev/i2c-*`)
- `--dry-run` выводит план, но не трогает SD-карту
- При попытке записать на примонтированное устройство — ошибка

12.10 Вариант 8. Система мониторинга со звуковой сигнализацией

12.10.1 Описание

Расширение варианта №1 (Метеостанция). При превышении порогового значения GUI автоматически воспроизводит звуковую сирену. Ха-

рактик звука настраивается для каждого измеряемого параметра индивидуально. Оператор может квитировать тревогу кнопкой.

12.10.2 Цепочка

BME280 —I2C→ Arduino —SPI→ Lichee —MQTT→ ПК (GUI + динамик)

12.10.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1.
2. **Lichee — SPI-приём + MQTT-публикация (лаб. №7, №12).** Идентично варианту №1.
3. **ПК — GUI-приложение.** Все элементы варианта №1 плюс:
 - Поля ввода порогов (T_max, H_max, P_min, P_max)
 - Настройки сирены для каждого параметра: тип звука (прерывистый быстрый/медленный/непрерывный/WAV-файл), частота тона (Гц)
 - Автоматическое воспроизведение сирены при превышении порога
 - Кнопка «Отключить сирену» (квитирование)
 - Лог тревог (таблица: Время, Параметр, Значение, Порог) + экспорт в CSV

Генерация звука — через стандартную библиотеку Python wave (без дополнительных зависимостей), воспроизведение через aplay.

12.10.4 Ключевые точки проверки

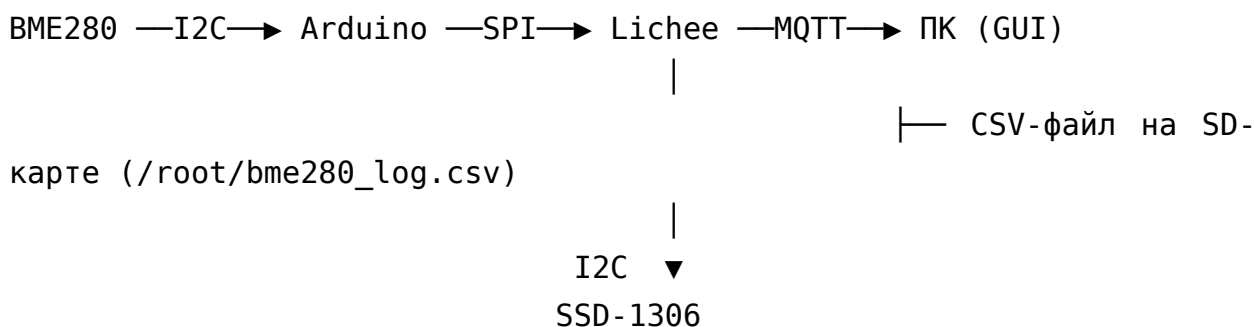
- При превышении порога GUI подсвечивает параметр красным и издаёт звуковую сирену
- Характер звука разный для разных параметров
- Кнопка «Отключить сирену» прекращает звук
- Лог тревог пополняется при каждом превышении; экспорт в CSV — корректный

12.11 Вариант 9. Самописец данных с хранением на SD-карте (Data Logger)

12.11.1 Описание

Собрать систему, в которой данные с датчика BME280 накапливаются на SD-карте одноплатника и одновременно доступны для live-мониторинга через MQTT. GUI на ПК отображает текущие значения и позволяет загрузить историю измерений.

12.11.2 Цепочка



12.11.3 Требования

1. **Arduino + датчик (лаб. №11).** Прошивка идентична варианту №1: читает BME280 по I2C, передаёт три float по SPI-slave на Lichee.
2. **Lichee — SPI-приём + логгер + MQTT (лаб. №7, №8, №12).** Python-скрипт:
 - Принимает данные от Arduino через /dev/spidev1.0
 - Дописывает строку в CSV-файл /root/bme280_log.csv:
2026-06-24T12:34:56,25.1,48.3,1004.5
 - Параллельно публикует текущие данные в MQTT-топик sensors/bme280 (JSON)
 - Дублирует текущие значения на OLED SSD-1306
3. **ПК — GUI-приложение.** Окно с элементами:

- Текущие значения (температура, влажность, давление)
- График реального времени (данные из MQTT)
- Кнопка «Загрузить историю» — по SCP или через HTTP качает CSV-файл с Lichee и отображает историю на отдельном графике
- Строка статуса: общее количество записей в логе, дата первой и последней записи

12.11.4 Ключевые точки проверки

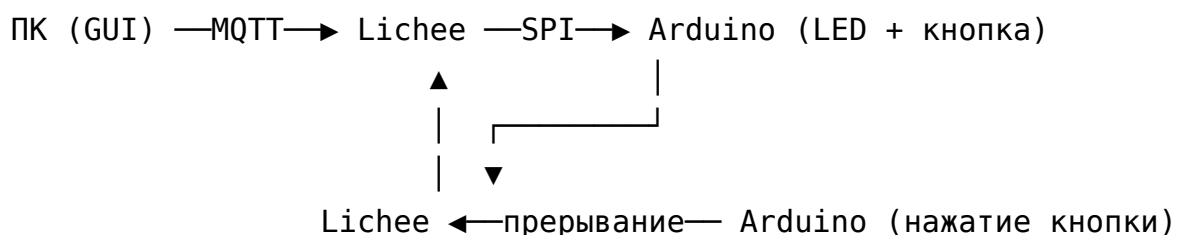
- Данные пишутся в CSV непрерывно, даже если GUI не запущен (проверить: запустить логгер, отключить ПК на минуту, снова подключиться — история сохранена)
- При перезагрузке Lichee файл лога сохраняется (SD-карта)
- График истории загружается корректно, временные метки соответствуют реальным
- Текущие значения на OLED обновляются синхронно с MQTT-поток

12.12 Вариант 10. Игра на реакцию

12.12.1 Описание

Реализовать игру на измерение времени реакции человека. ПК отправляет команду начала раунда. Lichee ждёт случайную задержку, затем зажигает светодиод на Arduino и одновременно публикует метку времени в MQTT. Игрок должен как можно быстрее нажать кнопку на Arduino, после чего вычисляется время реакции и отображается на GUI ПК.

12.12.2 Цепочка



|
▼ MQTT
ПК (GUI)

12.12.3 Требования

1. **Arduino — SPI-slave + прерывание (лаб. №10, №11).** Прошивка:

- Режим SPI-slave: принимает однокбайтовую команду от Lichee
- Команда 0x01 — зажечь LED на пине 5
- Команда 0x00 — погасить LED
- Кнопка подключена к пину 2 (прерывание FALLING): при нажатии отправляет байт 0xFF в SPI-буфер (мастер при следующей транзакции считает его)

2. **Lichee — ведущий игры (лаб. №12).** Python-скрипт, подписывающийся на MQTT-топик game/start:

- При получении {"action": "round"}:
 - Ждёт случайную задержку 1–5 секунд
 - Отправляет 0x01 по SPI (зажечь LED на Arduino)
 - Фиксирует время t0 = time.time()
 - Публикует {"event": "go", "timestamp": t0} в game/events
 - Начинает циклически опрашивать SPI (короткие транзакции с частотой ~100 Гц) для получения байта от Arduino
 - При получении 0xFF от Arduino фиксирует время реакции dt = time.time() - t0 и публикует в game/result
 - Отправляет 0x00 (погасить LED)

3. **ПК — GUI-приложение.** Окно с элементами:

- Кнопка «Новый раунд» — отправляет {"action": "round"} в game/start
- Крупное отображение времени реакции (мс) после завершения раунда
- Таблица последних 10 попыток (№, время реакции, результат)
- Лучший результат (минимальное время реакции за сессию)
- Индикатор состояния: «Ждите...», «ЖМИ!», «Результат»

12.12.4 Ключевые точки проверки

- Задержка перед зажиганием LED случайна и разная в каждом раунде
 - Время реакции измеряется в миллисекундах (типичные значения 150–400 мс)
 - Таблица результатов обновляется после каждого раунда
 - При нажатии кнопки до зажигания LED (фальстарт) фиксируется как ошибка и не засчитывается
 - GUI корректно отображает все состояния игры
-

12.13 Контрольные вопросы

1. Какую роль в итоговом проекте играет MQTT-брокер Mosquitto и почему он выбран в качестве транспортного протокола?
2. Почему в заданиях на связку Arduino→Lichee используется SPI, а не UART или I2C? Какие преимущества это даёт?
3. Каковы основные архитектурные паттерны, повторяющиеся во всех вариантах итогового задания?
4. Какие меры следует предусмотреть для обеспечения надёжности распределённой системы при возможных отказах компонентов?
5. Как организовать тестирование и отладку распределённой системы, когда все компоненты работают на разном оборудовании?

12.14 Порядок сдачи

1. Студент получает один из десяти вариантов задания
2. Разрабатывает и отлаживает все компоненты на оборудовании стенда
3. Демонстрирует преподавателю полностью работающую систему
4. Предоставляет исходный код всех компонентов (Arduino .ino, Python-скрипты Lichee, GUI-приложение ПК, Makefile/инструкцию по сборке для С-частей)
5. Проходит устную защиту: объяснение архитектуры, протоколов обмена, выбранных решений

6. Сформировать отчёт о выполнении задания .doc и **выслать на почту преподавателя до обозначенного срока**